Machine Intelligence

5. Abstraction Heuristics, Pattern Databases

Daniel Fišer

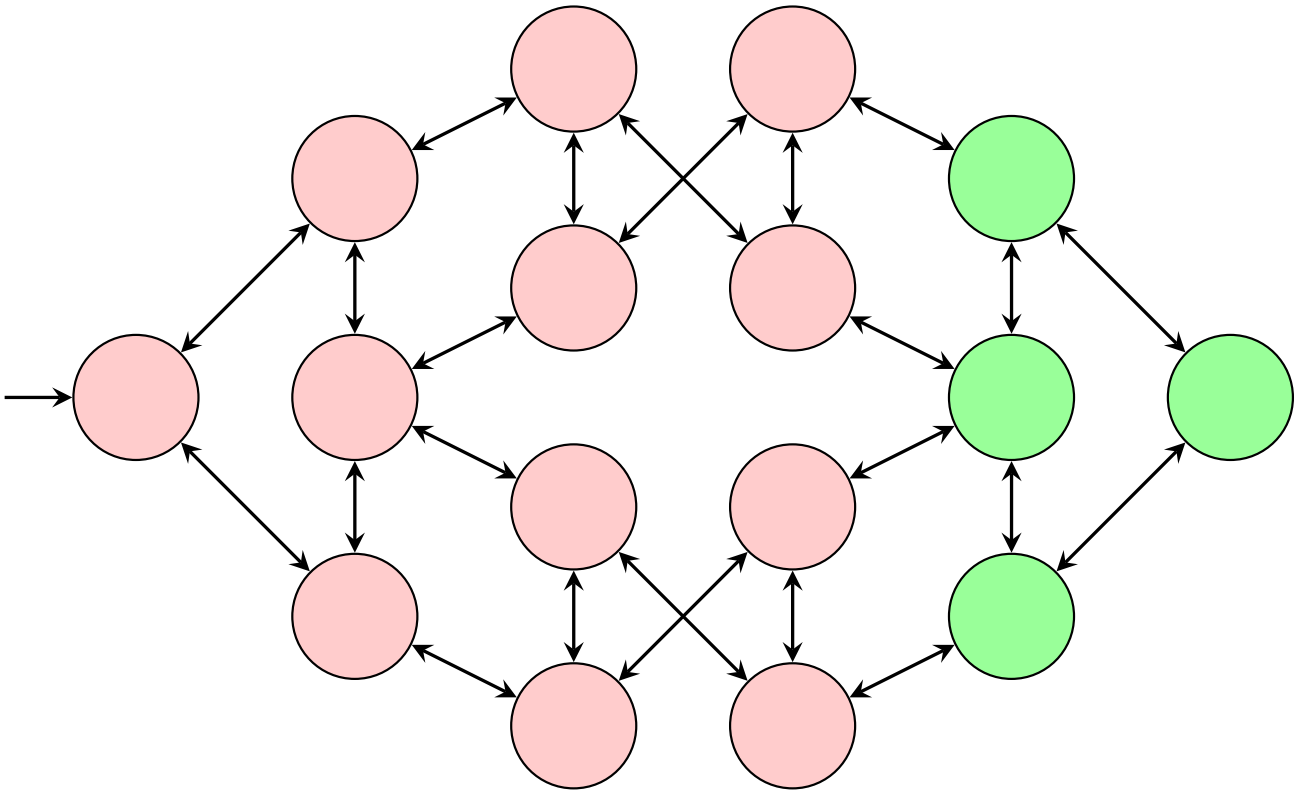
AALBORG UNIVERSITY

Fall 2025

Agenda

1. Introduction
2. Abstraction Basics
3. Pathological Abstractions
4. Pattern Databases (PDB)
5. Orthogonal Pattern Databases
6. Summary

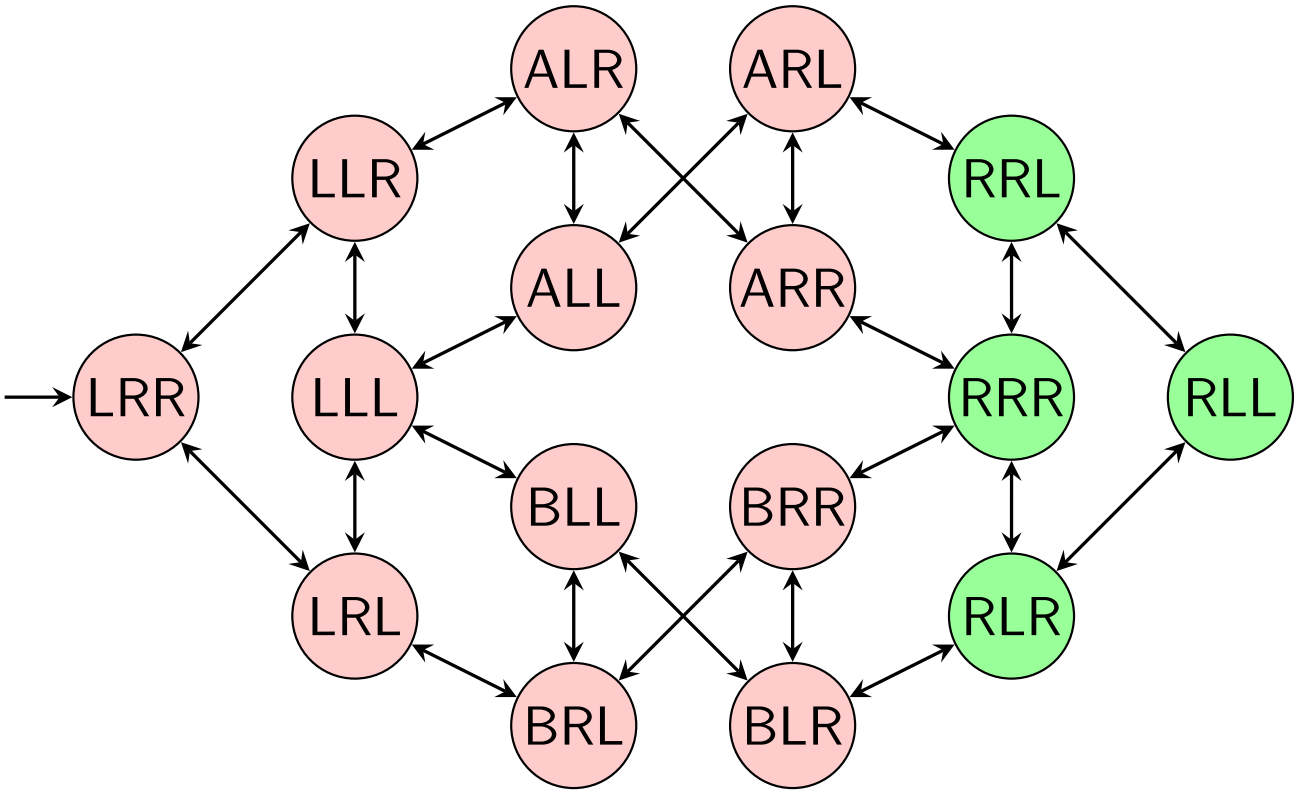
This is How We'll Depict State Spaces Here



$\rightsquigarrow$ To reduce clutter, the figures usually omit action names, and collapse multiple transitions between the same pair of states.

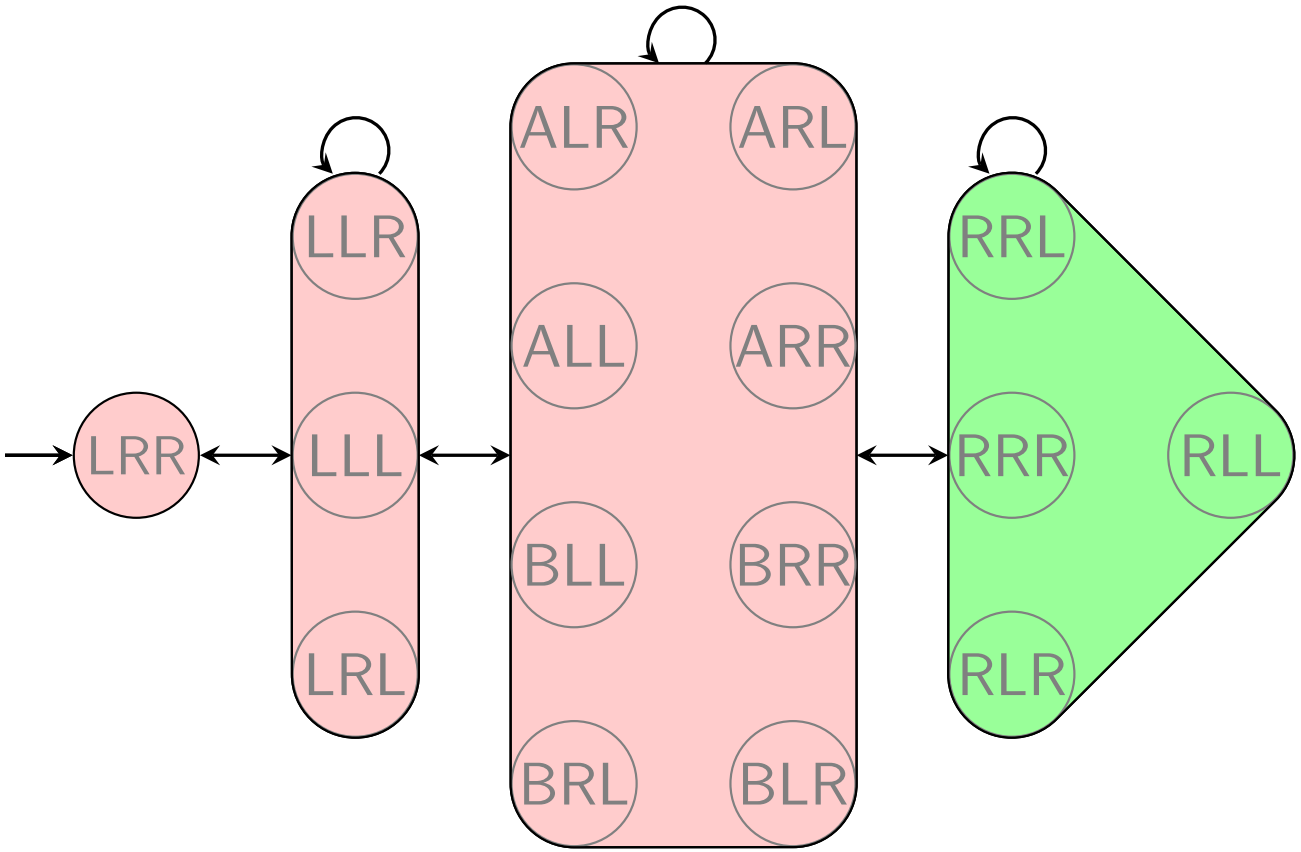
Abstractions in a Nutshell: Example

Concrete state space: (of "Logistics mal anders", see later)



Abstractions in a Nutshell: Example

Abstract state space: (of "Logistics mal anders", see later)



Abstractions in a Nutshell

$\rightsquigarrow$ Abstracting a transition system means dropping some distinctions between states, while preserving all transitions and goal states.

- An abstraction of a transition system Θ is defined by a function α (the abstraction mapping), mapping states to abstract states (also block states).
- If α maps states s and t to the same abstract state, then s and t are not distinguished anymore (they are equivalent under α).
- The abstract state space Θ^α on the image of α is defined by homomorphically mapping the goal states and transitions of Θ to Θ^α, thus preserving all solutions.
- h^* in Θ^α is the abstraction heuristic h^α in Θ. As we preserve all solutions, h^α is admissible.

Agenda

- **Abstraction Basics**

  ⇝ Formal definition of abstractions and their associated structures; proving their basic properties.

- **Practical vs. Pathological Abstractions:** Interesting rim cases . . .

  ⇝ We briefly illuminate basic practical issues, through a number of examples illustrating "how not to do it".

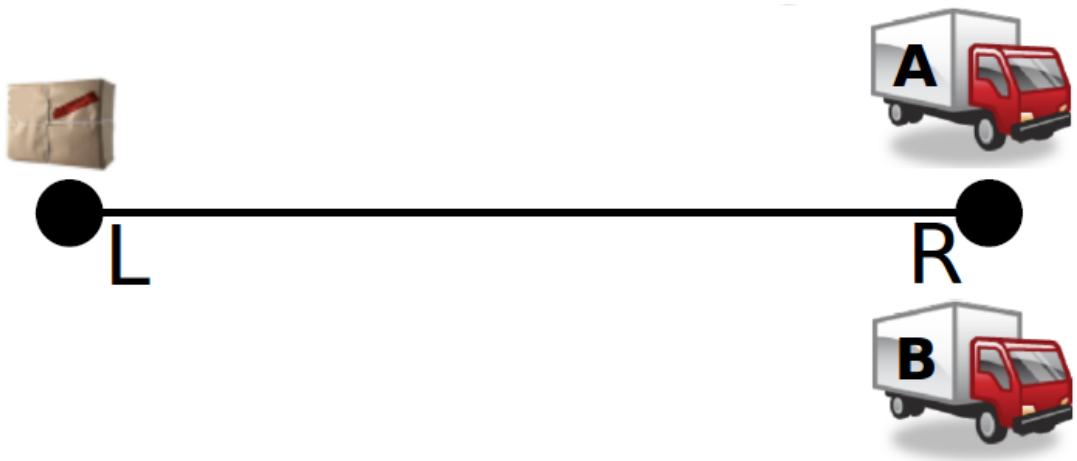
- **Pattern Databases:** One instance of abstraction heuristics

  ⇝ We show how to efficiently construct abstractions by disregarding some variables.

- **Additive Pattern Database Heuristics:** Orthogonal patterns allow to sum heuristics

  ⇝ We introduce a simple criterion allowing to admissibly sum up several PDB heuristics.

"Logistics mal anders": *One* Package, *Two* Trucks



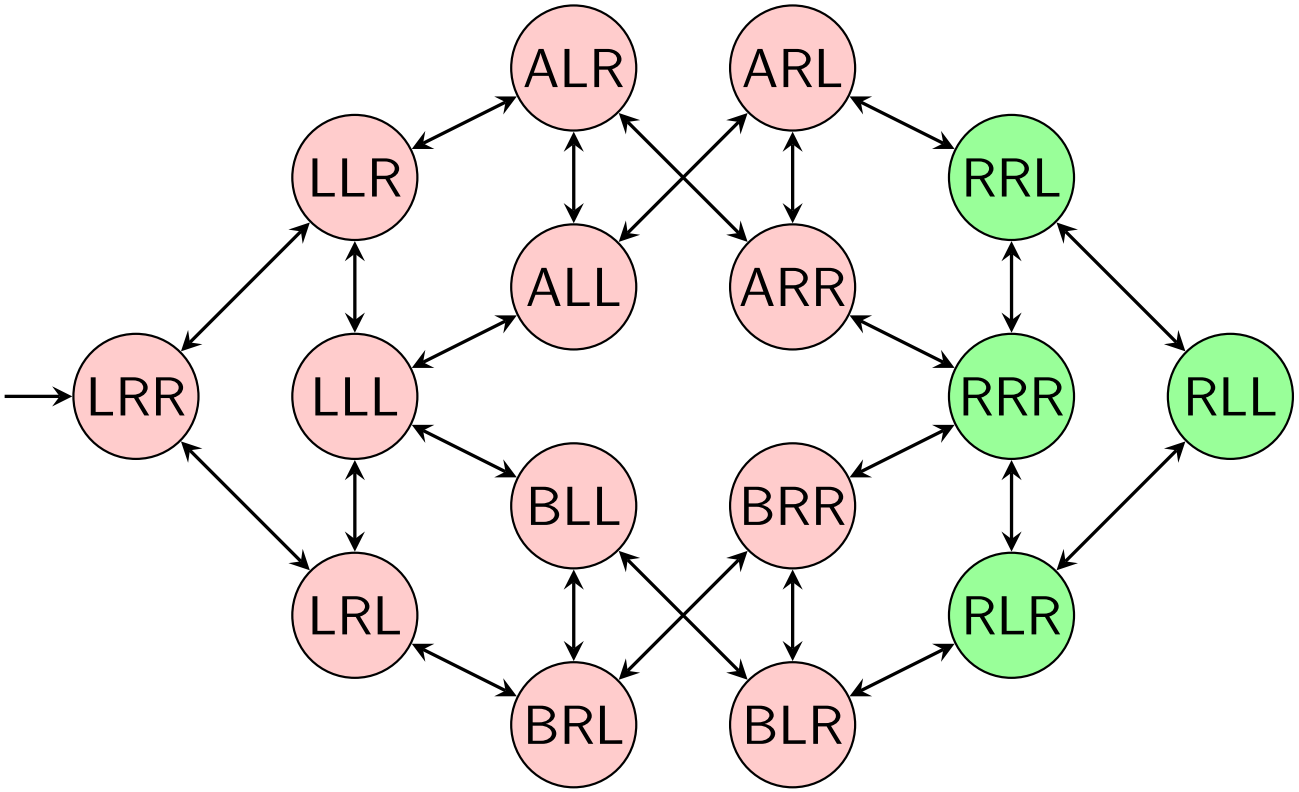
$\mathcal{V} = \{p, t_A, t_B\}$ with $D_p = \{L, R, A, B\}$ and $D_{t_A} = D_{t_B} = \{L, R\}$.

$$\begin{aligned}\mathcal{A} = &\{\mathit{pickup}(x, y) \mid x \in \{A, B\}, y \in \{L, R\}\}\\ &\cup \{\mathit{drop}(x, y) \mid x \in \{A, B\}, y \in \{L, R\}\}\\ &\cup \{\mathit{move}(x, y, y') \mid x \in \{A, B\}, y, y' \in \{L, R\}, y \neq y'\}, \textit{with}\end{aligned}$$

- $\mathrm{pre}_{pickup(x,y)}$: $t_x = y, p = y$; $\mathrm{eff}_{pickup(x,y)}$: $p = x$;
- $\mathrm{pre}_{drop(x,y)}$: $t_x = y, p = x$; $\mathrm{eff}_{drop(x,y)}$: $p = y$;
- $\mathrm{pre}_{move(x,y,y')}$: $t_x = y$; $\mathrm{eff}_{move(x,y,y')}$: $t_x = y'$.

$I = \{p = L,\ t_A = R,\ t_B = R\}$. $G = \{p = R\}$.

The State Space of "Logistics mal anders"



- State $p = x$, $t_A = y$, $t_B = z$ is depicted as xyz.
- Action names not shown. For example, the transition from LLL to ALL is induced by $pickup(A, L)$.

Abstractions

Definition (Abstraction)

Let $\Theta_\Pi = (\mathcal{S}, \mathcal{A}, \text{cost}, \mathcal{T}, I, \mathcal{S}_G)$ be a state space. An abstraction of Θ is a surjective function $\alpha : S \mapsto S^\alpha$. The abstract state space induced by α, written Θ^α, is the (possibly non-deterministic) state space $\Theta^\alpha = (\mathcal{S}^\alpha, \mathcal{A}, \text{cost}, \mathcal{T}^\alpha, I^\alpha, \mathcal{S}^\alpha_G)$ such as

- $I^\alpha = \alpha(I)$ (preserve the initial state)
- $\mathcal{S}^\alpha_G = \{\alpha(s) \mid s \in S^G\}$ (preserve goal states)
- $\mathcal{T}^\alpha = \{(\alpha(s), a, \alpha(t)) \mid (s, a, t) \in T\}$ (preserve transitions)

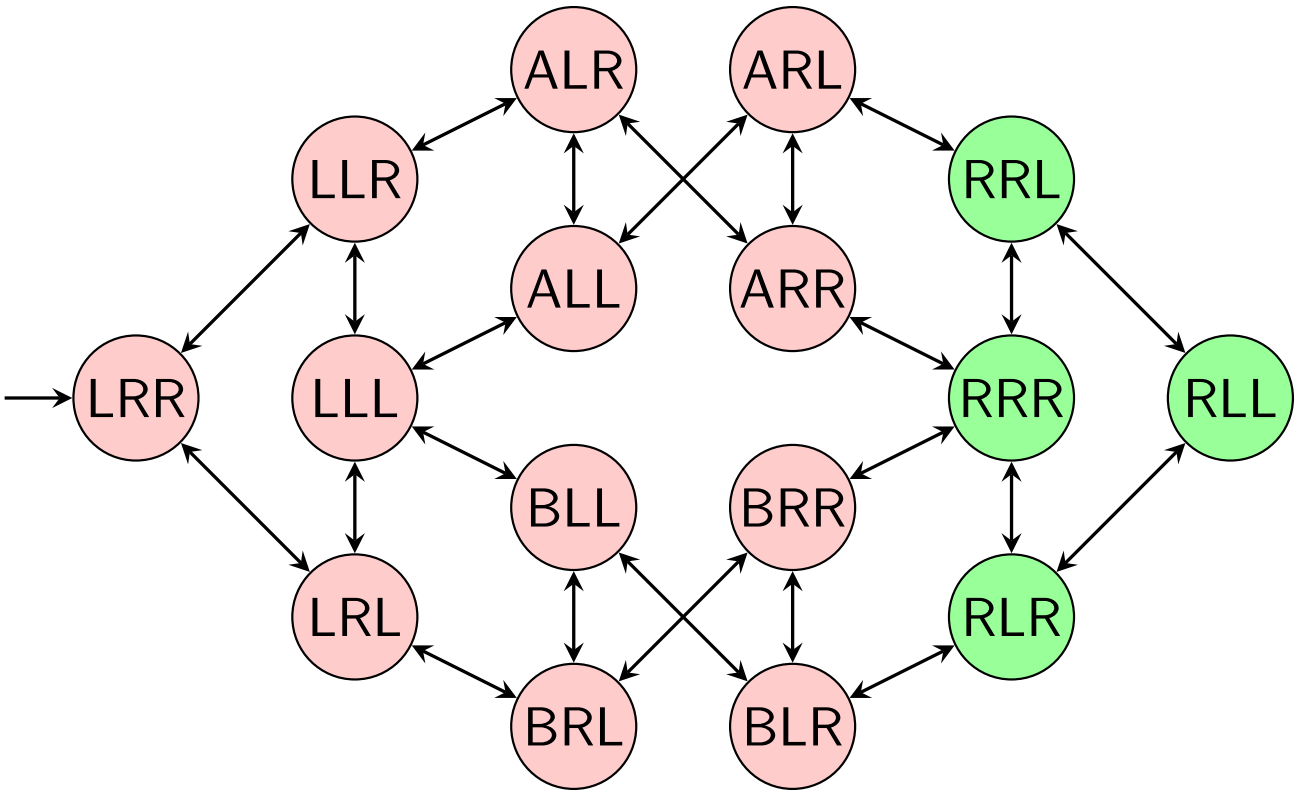
The size of the abstraction is the number $|S^\alpha|$ of abstract states.

$\rightsquigarrow$ Θ is called the concrete state space. Similarly: concrete/abstract state space, concrete/abstract transition, etc.

$\rightsquigarrow$ Why do we require α to be surjective? So that Θ^α does not contain superfluous states.

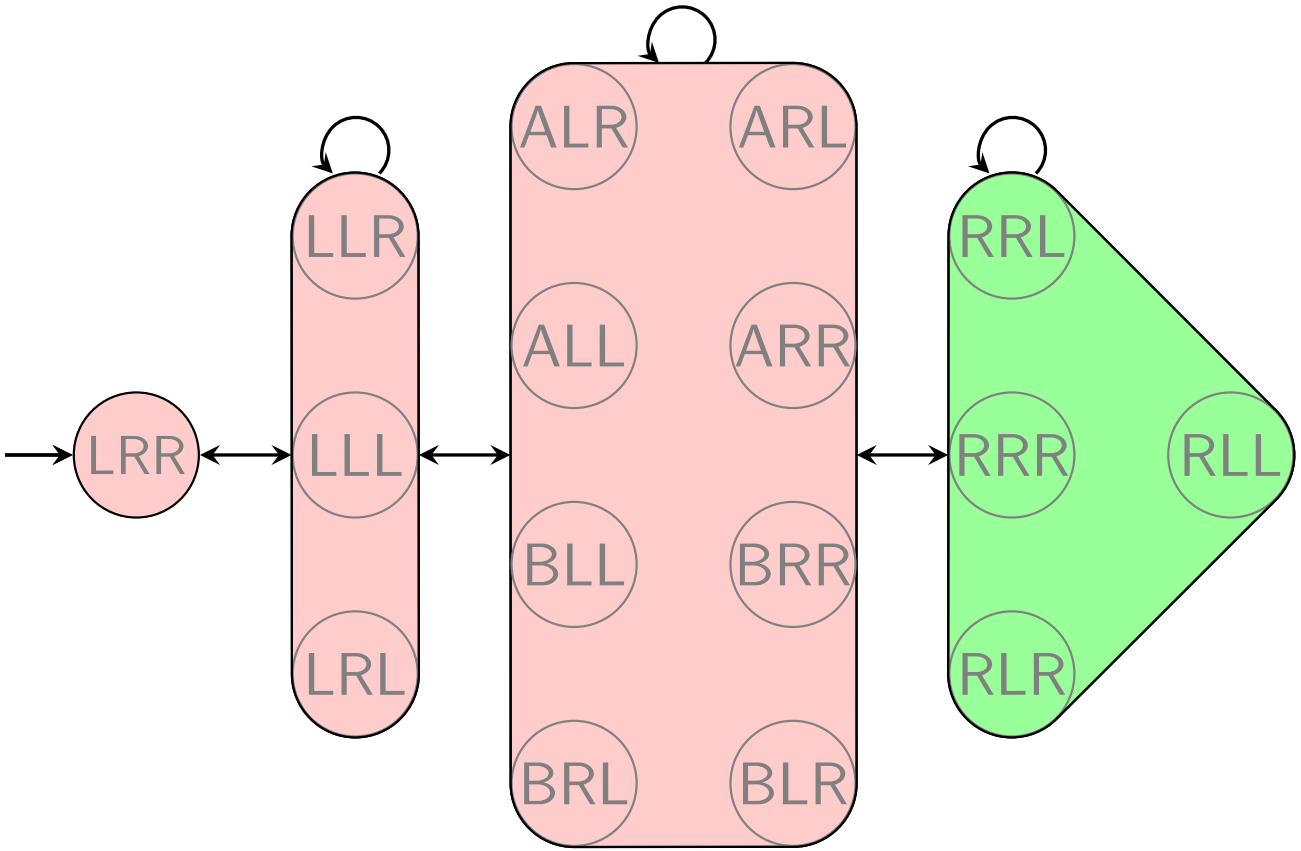
Abstractions: "Logistics mal anders"

Concrete state space:



Abstractions: "Logistics mal anders"

Abstract state space:



⇝ A transition between concrete states is "spurious" if it exists in the abstract but not in the concrete state space. Example here? We can go in a single step from LRR to LLL; same for ALR to RRR etc.

Abstraction Heuristics

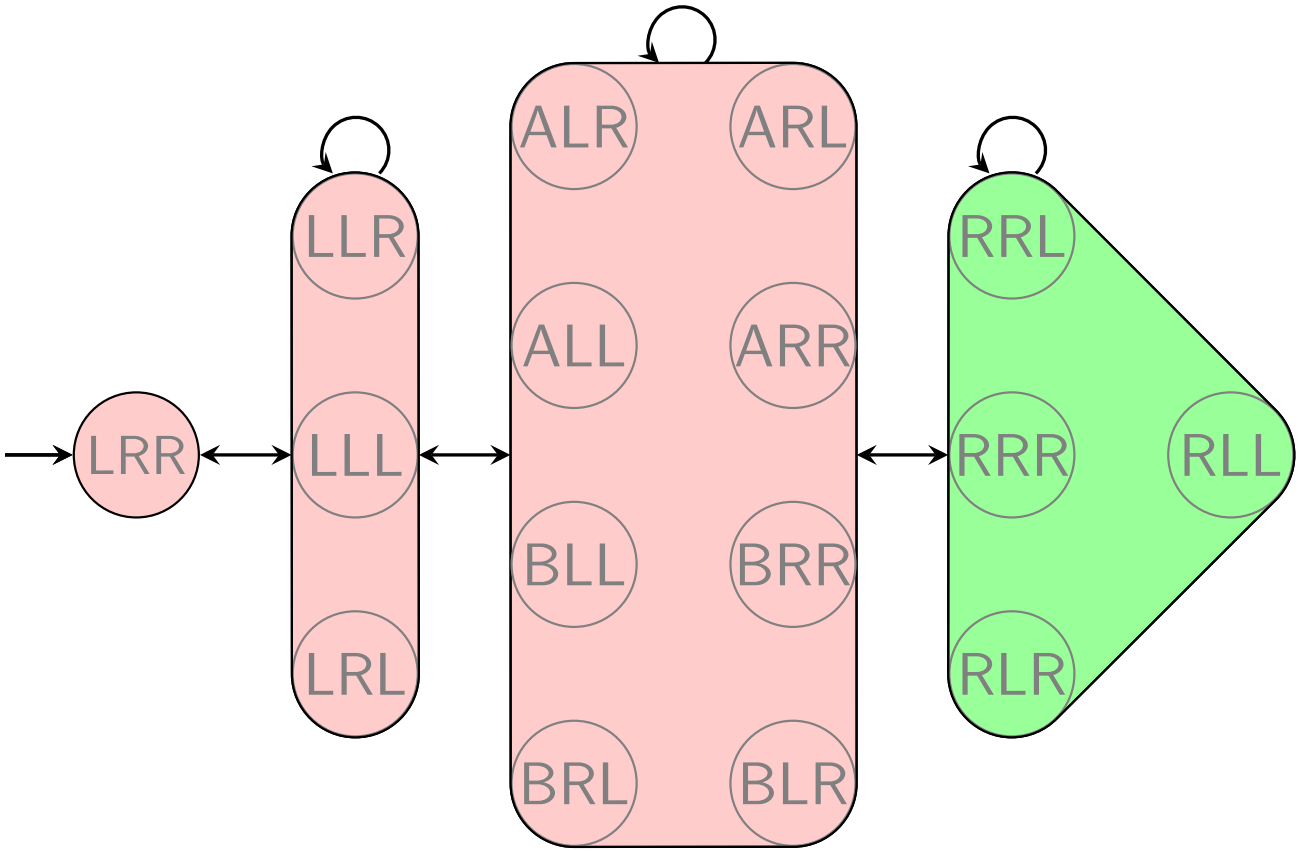
Definition (Abstraction Heuristic)

Let $\Theta = (S, A, c, T, I, S^G)$ be a state space, and let α be an abstraction of Θ. The abstraction heuristic induced by α, written h^α, is the heuristic function $h^\alpha : S \mapsto \mathbb{N}_0 \cup \{\infty\}$ which maps each state $s \in S$ to $h^*_{\Theta^\alpha}(\alpha(s))$, i.e., to the cost of the optimal $\alpha(s)$-plan in the abstract state space Θ^α.

To compute $h^\alpha(s)$, we need to:

- map s to $\alpha(s)$
  $\rightsquigarrow$ we need to come up with an abstraction that allows fast computation of $\alpha(s)$
- compute h^* for $\alpha(s)$ in Θ^α, i.e., $h^*_{\Theta^\alpha}(\alpha(s))$
  $\rightsquigarrow$ we need to come up with α so that we can pre-compute h^* for all abstract states, and then find $h^*_{\Theta^\alpha}(\alpha(s))$ in a look-up table.

Abstraction Heuristics: "Logistics mal anders"



$$h^{\alpha}(\{p = L, t_A = R, t_B = R\}) = 3 \neq h^{*}(\{p = L, t_A = R, t_B = R\}) = 4$$

Abstraction Heuristics: Properties

Theorem (h^α is Consistent and Admissible)

Let Θ be a state space, and let α be an abstraction of Θ. Then h^α is consistent, and therefore also admissible and safe.

Proof. Let $\Theta = (S, A, c, T, I, S^G)$ and $\Theta^\alpha = (S^\alpha, A, c, T^\alpha, I^\alpha, S^{\alpha G})$.

Since $\alpha(s) \in \mathcal{S}_G^\alpha$ for every $s \in \mathcal{S}_G$, we have that $h^\alpha(s) = 0$, i.e., h^α is a well-defined heuristic function.

For consistency, we need to show that $h^\alpha(s) \leq h^\alpha(t) + c(a)$ whenever $(s, a, t) \in \mathcal{T}$. By definition, $h^\alpha(s) = h^*_{\Theta^\alpha}(\alpha(s))$ and $h^\alpha(t) = h^*_{\Theta^\alpha}(\alpha(t))$, so we need to show that $h^*_{\Theta^\alpha}(\alpha(s)) \leq h^*_{\Theta^\alpha}(\alpha(t)) + c(a)$. Since (s, a, t) is a concrete transition, by definition of abstractions we have an abstract transition $(\alpha(s), a, \alpha(t))$ in Θ^α. Therefore, since h^* is always consistent and thus also in Θ^α, it immediatelly follows that $h^*_{\Theta^\alpha}(\alpha(s)) \leq h^*_{\Theta^\alpha}(\alpha(t)) + c(a)$.

Quiz: Real vs. Abstract Plans

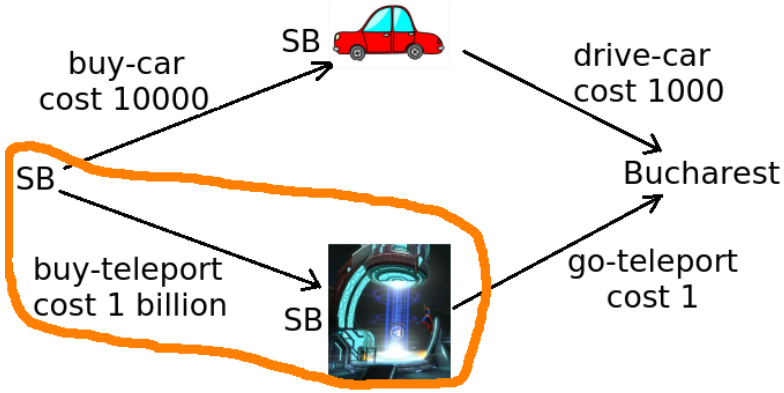
$\rightsquigarrow$ Are optimal abstract plans just abstractions of optimal real plans?

The situation: Assume an FDR planning task Π, a state s, and an optimal plan π for s in Π. Say α is an abstraction, and say we obtain π^α from π by *removing all actions that induce self-loops in θ^α.*

Quiz

Is π^α necessarily an optimal abstract plan, i.e., $\mathrm{cost}(\pi^\alpha) = h^\alpha(s)$?

$\rightsquigarrow$ No! Spurious transitions may lead to "shortcuts" that do not correspond to an optimal real plan, or to any plan at all. **Example:**



Car: cost 10000 (buy) $+$ 1000 (go); teleport: cost 1 billion (buy) $+$ 1 (go). If α does not distinguish between SB and the state where we have the teleport, then SB has a spurious cost-1 transition to Bucharest, and the only optimal abstract plan uses that transition.

Which Abstractions Should We Use in Practice?

Conflicting Objectives

The eternal trade-off between accuracy and efficiency:

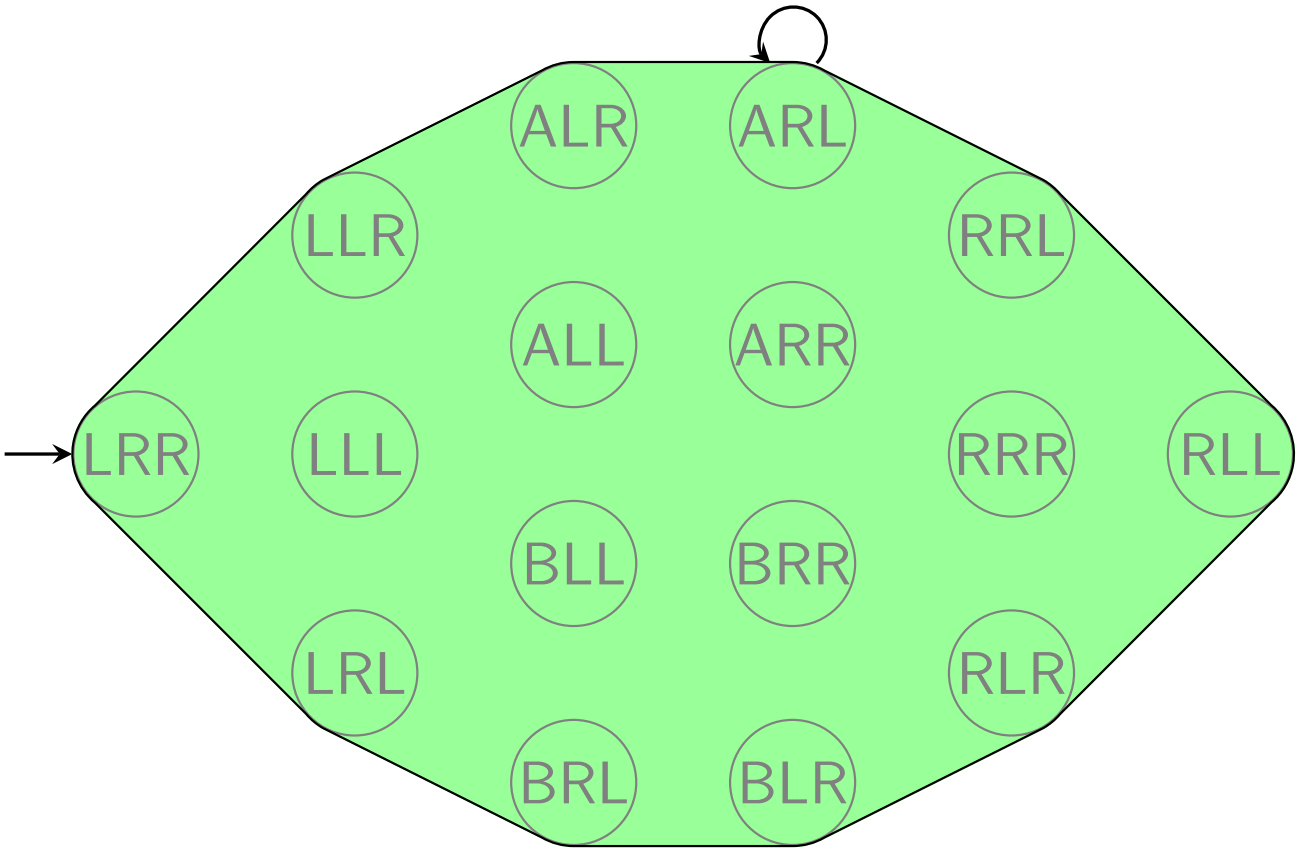
- We want to obtain an informative heuristic.
- We want to obtain a small computational overhead.

$\rightsquigarrow$ The abstraction function α is a very powerful parameter, allowing to travel the whole way between both extremes.

$\rightsquigarrow$ What do we mean by "small computational overhead"?

- **Fast computation of α:** For a given state s, the abstract state $\alpha(s)$ must be efficiently obtainable.
- **Few abstract states:** For a given abstract state $\alpha(s)$, the $h^\alpha(s) = h^*_{\Theta^\alpha}(\alpha(s))$ must be efficiently computable.

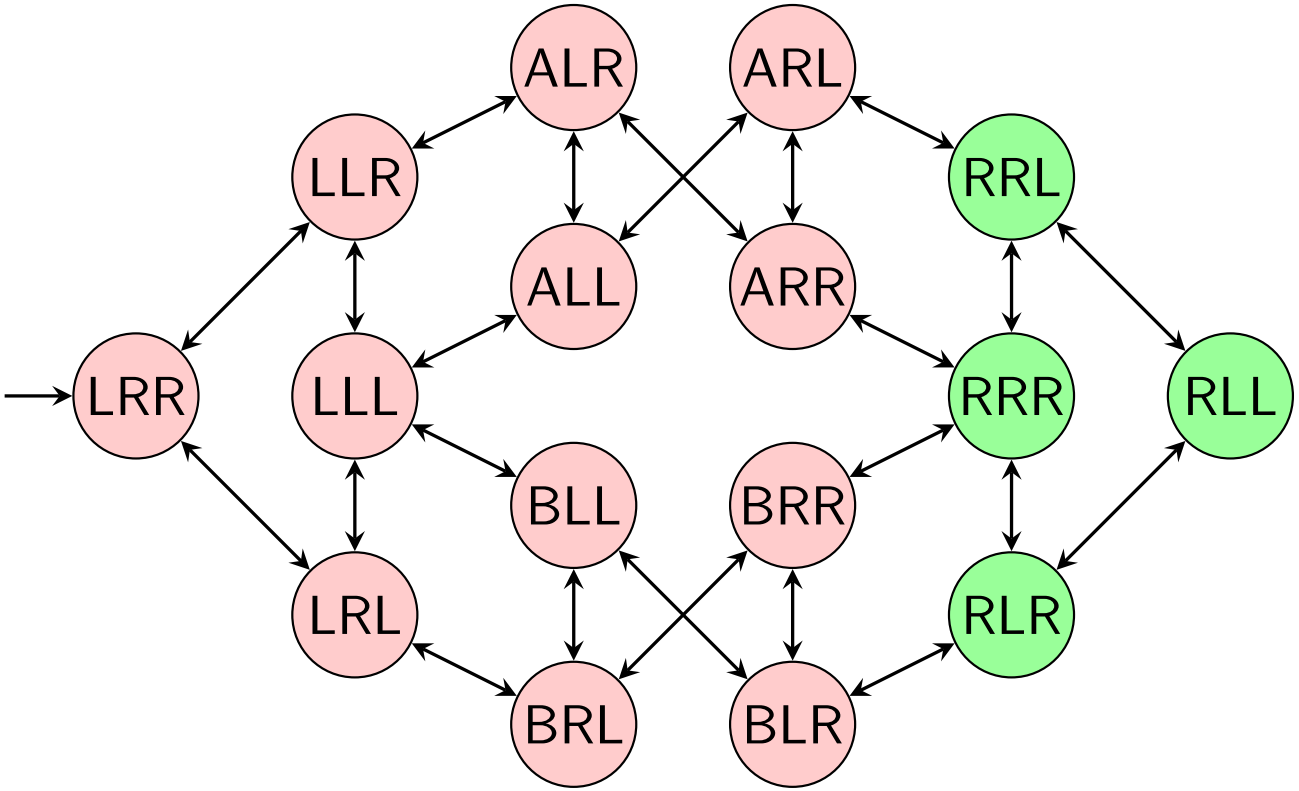
Pathological Case 1: One-State Abstraction



One-state abstraction: $\alpha(s) :=$ const.

+ Trivial to compute α and $h^*_{\Theta^\alpha}$, just one abstract state.
− Completely uninformative h^α.

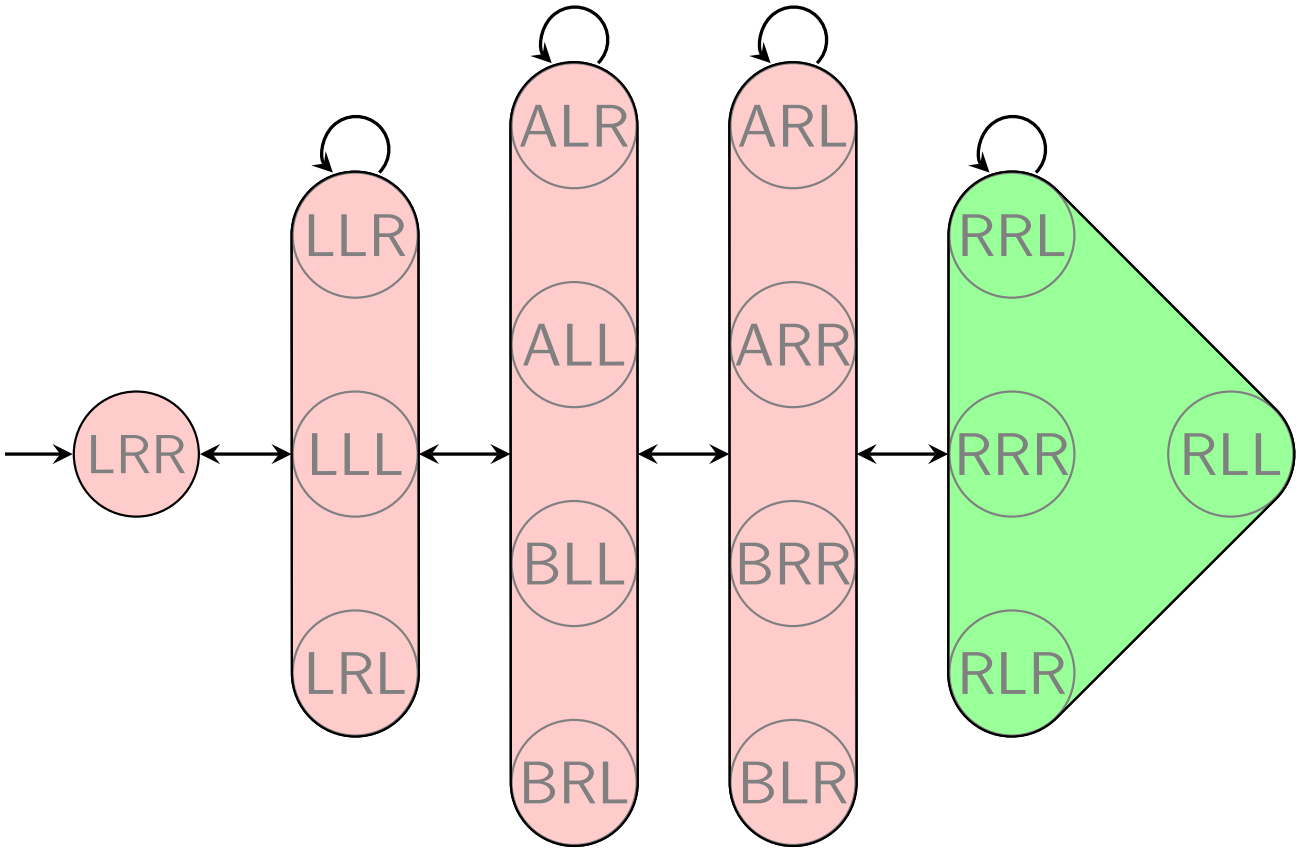
Pathological Case 2: Identity Abstraction



Identity abstraction: $\alpha(s) := s$.

+ $h^{\alpha} = h^{*}$, trivial to compute α.
− Computing $h^{*}_{\Theta^{\alpha}}$ = solving the optimal planning problem.

Pathological Case 3: Perfect Abstraction



Perfect abstraction: $\alpha(s) := h^*(s)$.

+ $h^\alpha = h^*$, few abstract states so easy to compute $h^*_{\Theta^\alpha}$.
− Computing α = solving the optimal planning problem.

So, How to Obtain *Non*-Pathological Abstractions?

Covered in this course:

- Pattern database heuristics [Culberson and Schaeffer (1998); Edelkamp (2001); Haslum *et al.* (2007)].

Not covered in this course:

- Domain Abstractions, obtained by aggregating values within state variable domains [Hernádvölgyi and Holte (2000)]. Generalizes pattern database heuristics.
- Cartesian Abstractions, where abstract states are characterized by cross-products of state-variable-domain-subsets [Seipp and Helmert (2013)]. Generalizes domain abstractions.
- Merge-and-shrink abstractions, where abstractions are iteratively obtained by cross-product (merging) and abstraction (shrinking) steps, starting at single-variable-abstractions [Dräger *et al.* (2006); Helmert *et al.* (2007); Katz *et al.* (2012); Helmert *et al.* (2014)]. Generalizes Cartesian abstractions.

Motivation for Pattern Database Heuristics

$\rightsquigarrow$ Pattern databases are a concrete method for designing abstraction functions α, and for computing the associated heuristic functions.

There's many good reasons to be considering PDBs:

- Pattern database (PDB) heuristics are the most commonly used class of abstraction heuristics outside planning (Games, mostly).
- PDBs are one of the most successful classes of abstraction heuristics in planning.
- PDBs have been an active research area from their inception, and still are an active research area today. (Theoretical properties, how to implement and use PDBs effectively, how to find good patterns, ...)
- For many search problems, pattern databases are the most effective admissible heuristics currently known.

Pattern Database Heuristics

"Pattern database heuristics" = Heuristics induced by a particular class of abstraction mappings, namely projections:

Definition (Projection, PDB Heuristic)

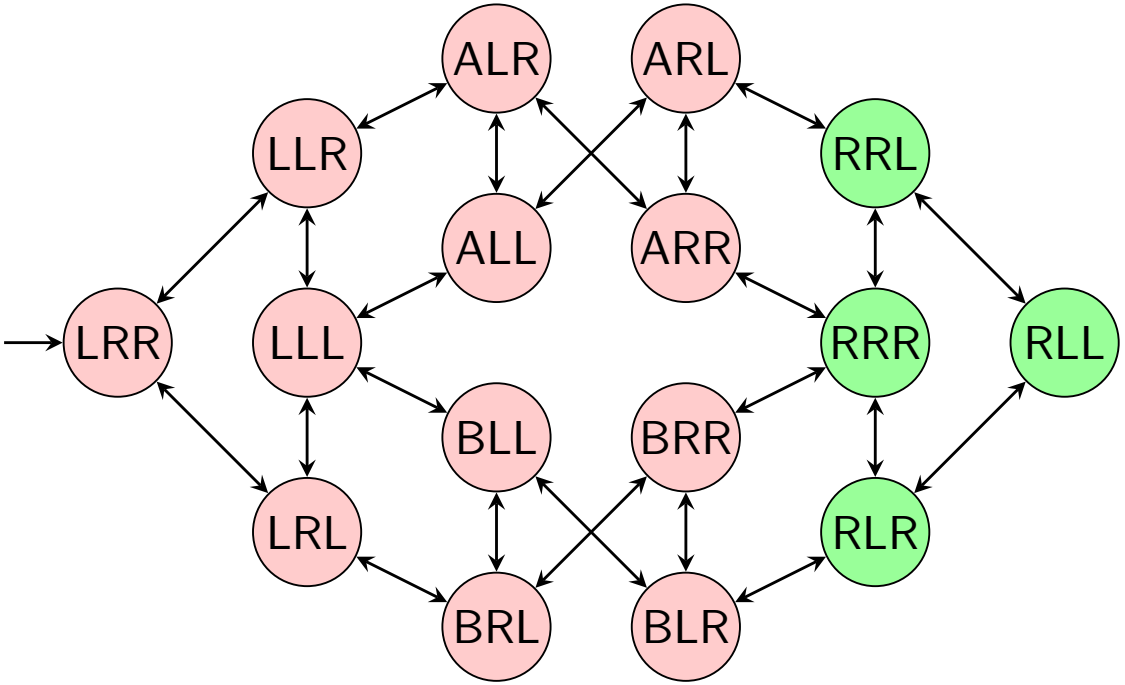
Let $\Pi = (\mathcal{V}, \mathcal{A}, \mathrm{cost}, I, G)$ be an FDR planning task with the state space $\Theta_\Pi = (\mathcal{S}, \mathcal{A}, \mathrm{cost}, \mathcal{T}, I, \mathcal{S}_G)$, and let $P \subseteq \mathcal{V}$.
For a partial state φ to V, by $\varphi|_P$ we denote the restriction of φ to P.
Let $\mathcal{S}^P$ be the set of partial states over P. The projection $\pi_P \colon \mathcal{S} \mapsto \mathcal{S}^P$ is defined by $\pi_P(s) := s|_P$.

$\rightsquigarrow$ π_P maps two states s_1 and s_2 to the same abstract state iff they agree on all variables in P.

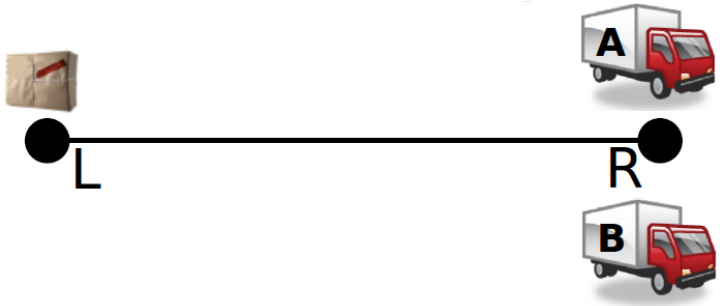
We refer to P as the pattern of π_P. The abstraction heuristic induced by π_P on Θ_Π is called a pattern database heuristic, short PDB heuristic. We write h^P as a short-hand for h^{π_P}, and we write Θ_Π^P or Θ^P as short-hands for $\Theta_\Pi^{\pi_P}$.

- h^P is usually stored in a lookup table called a pattern database (PDB).

"Logistics mal anders": State Space



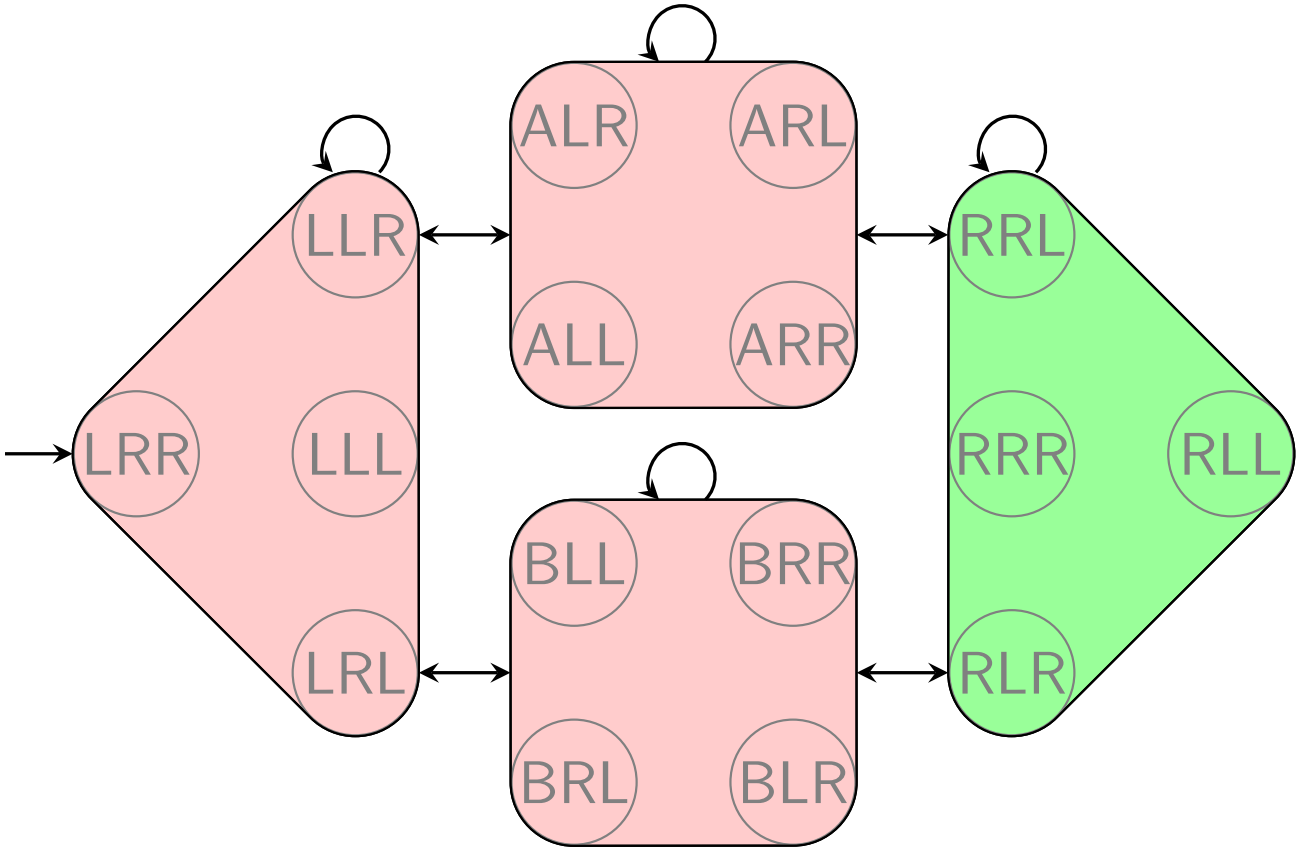
Logistics task with one package, two trucks, two locations:



- State variable package: $\{L, R, A, B\}$.
- State variable truck A: $\{L, R\}$.
- State variable truck B: $\{L, R\}$.

"Logistics mal anders": Projection 1

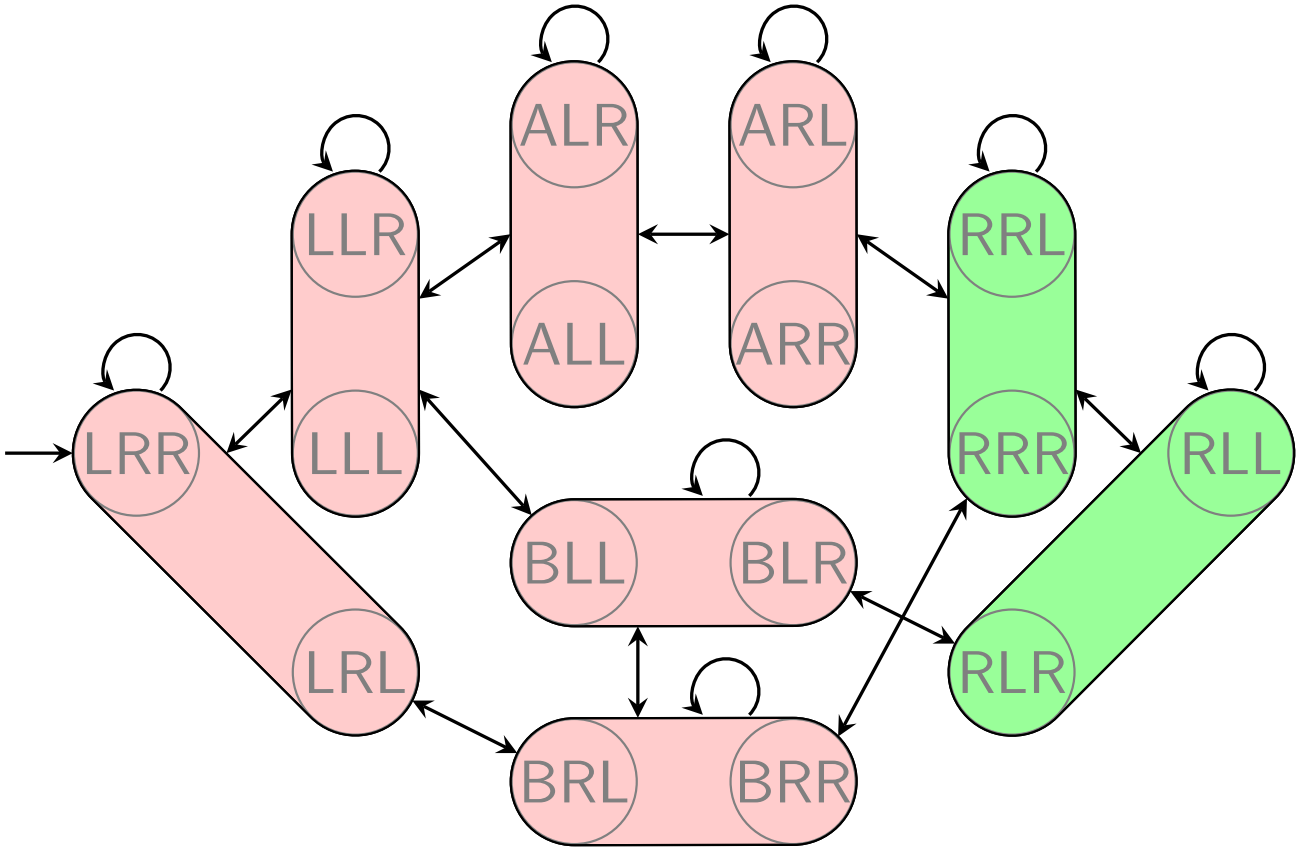
Abstraction induced by $\pi_{\{\text{package}\}}$:



$$h^{\{\text{package}\}}(\text{LRR}) = 2$$

"Logistics mal anders": Projection 2

Abstraction induced by $\pi_{\{\text{package},\text{truck A}\}}$:



$$h^{\{\text{package},\text{truck A}\}}(\text{LRR}) = 2$$

Larger Pattern = More Information

Theorem (Larger Patterns Induce More Informative Heuristics)

Let Π be an FDR planning task with variables $\mathcal{V}$ and states $\mathcal{S}$, and let $P' \subseteq P \subseteq \mathcal{V}$. Then $h^P(s) \geq h^{P'}(s)$ for every state $s \in \mathcal{S}$.

Proof Idea. $\pi_{P'}$ can be viewed as an abstraction of Θ^P. The rest follows from what we already know about admissibility of abstraction heuristics, i.e., h^* in $\Theta^{P'}$ is a lower-bound on h^* in Θ^P.

- And, of course, $h^{\mathcal{V}} = h^*$.
- Pattern size controls the trade-off between accuracy and computational cost.

I Give You Pattern, You Give Me Database!

Asssume: You are given a pattern P.

- How do you compute h^P?
- More precisely: How do you compute a data structure that efficiently represents the function $h^P(s)$, for all states s?

Here's how:

1. In a precomputation step, we compute an explicit graph representation for the abstract state space $\Theta_{\Pi}^{\pi_P}$, and compute the abstract remaining cost for every abstract state.
2. We store the abstract remaining cost in a table. During search, we use lookup in this table to implement h^P.

(I) Precomputation Step: It's Not That Easy

Let Π be a planning task and P a pattern. Let $\Theta = \Theta_\Pi$ and $\Theta' = \Theta_\Pi^{\pi_P}$. We want to compute a graph representation of Θ'.

So, what's the issue?

- Θ' is defined through a function on Θ:
  - Each concrete transition induces an abstract transition, each concrete goal state induces an abstract goal state.
- In principle, we can compute Θ' by iterating over all transitions/goal states of Θ. BUT:
  - This would take time $\Omega(\|\Theta\|)$.
  - Which comes down to solving the original (concrete, not abstract) planning task in the first place, using blind search.

$\rightsquigarrow$ We need a way of computing Θ' in time polynomial in $\|\Pi\|$ and $\|\Theta'\|$.

(I) Precomputation Step: Here's How To

Definition (Syntactic Projection)

Let $\Pi = (\mathcal{V}, \mathcal{A}, \mathrm{cost}, I, G)$ be an FDR planning task, and let $P \subseteq \mathcal{V}$. The syntactic projection of Π to P is the FDR planning task $\Pi|_P = (P, \mathcal{A}|_P, c, I|_P, G|_P)$ where $\mathcal{A}|_P := \{a|_P \mid a \in \mathcal{A}\}$ with $\mathrm{pre}_{a|_P} = (\mathrm{pre}_a)|_P$ and $\mathrm{eff}_{a|_P} = (\mathrm{eff}_a)|_P$.

$\rightsquigarrow$ $\Pi|_P$ removes the variables outside P from all constructs in the planning task description Π.

Theorem (Syntactic Projection is Equivalent to Projection)

Let Π be an FDR planning task, and let $P \subseteq \mathcal{V}$. Then $\Theta_{(\Pi|_P)}$ is identical to $\Theta_\Pi^{\pi_P}$ except that actions a in $\Theta_\Pi^{\pi_P}$ correspond to actions $a|_P$ in $\Theta_{(\Pi|_P)}$.

Proof. Easy from definition.

$\rightsquigarrow$ The state space of the syntactic projection is (modulo action renaming) the same as the abstract state space of the projection.

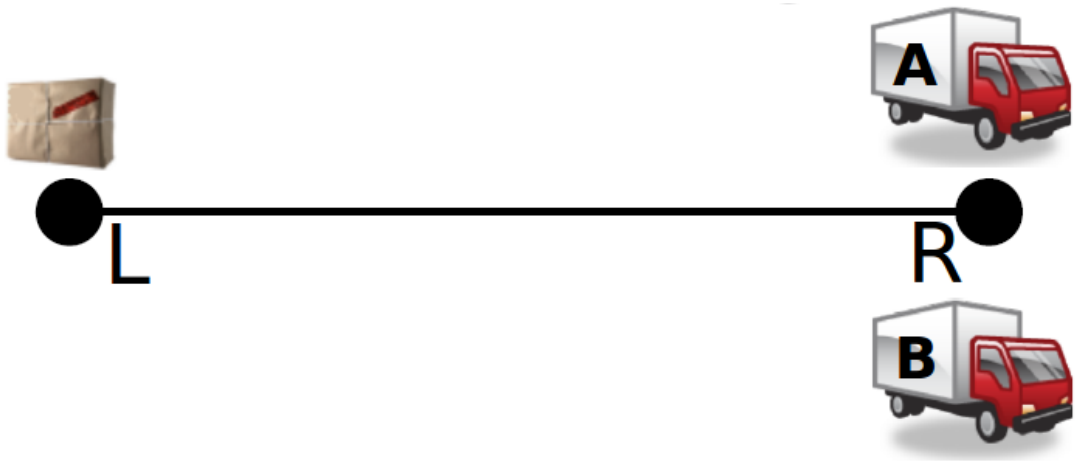
(I) Precomputation Step: Here's How To, ctd.

Using the Theorem on the previous slide, we can compute pattern databases for FDR tasks Π and patterns P as follows:

function `computePDB`$(\Pi = (\mathcal{V}, \mathcal{A}, \mathrm{cost}, I, G)$, *pattern* $P \subseteq \mathcal{V})$ **returns** a PDB
  $\Pi' \leftarrow \Pi|_P$ `/* Compute syntactic projection */`
  Compute the reachable part of $\Theta_{\Pi'}$ by a complete forward search;
  In the explicit graph $\Theta_{\Pi'}$, add a new node x with a 0-cost incoming edge from every goal node;
  Run Dijkstra starting from x traversing edges backwards—this results in $h^*_{\Theta_{\Pi'}}$ for all abstract states in $\Theta_{\Pi'}$;
  return *A table mapping abstract states to their* $h^*_{\Theta_{\Pi'}}$*-value*

$\rightsquigarrow$ This algorithm runs in time and space polynomial in $\|\Pi\| + \|\Theta_{\Pi'}\|$.

(I) Example



$\mathcal{V} = \{p, t_A, t_B\}$ with $D_p = \{L, R, A, B\}$ and $D_{t_A} = D_{t_B} = \{L, R\}$.

$$\begin{aligned}\mathcal{A} = &\{pickup(x, y) \mid x \in \{A, B\}, y \in \{L, R\}\}\\ &\cup \{drop(x, y) \mid x \in \{A, B\}, y \in \{L, R\}\}\\ &\cup \{move(x, y, y') \mid x \in \{A, B\}, y, y' \in \{L, R\}, y \neq y'\}, \textit{with}\end{aligned}$$

- $\text{pre}_{pickup(x,y)}$: $t_x = y, p = y$; $\text{eff}_{pickup(x,y)}$: $p = x$;
- $\text{pre}_{drop(x,y)}$: $t_x = y, p = x$; $\text{eff}_{drop(x,y)}$: $p = y$;
- $\text{pre}_{move(x,y,y')}$: $t_x = y$; $\text{eff}_{move(x,y,y')}$: $t_x = y'$.

$I = \{p = L,\ t_A = R,\ t_B = R\ \}$. $G = \{p = R\}$.

(I) Example

Projection to $P = \{p\}$:

$\mathcal{V} = \{p\}$ with $D_p = \{L, R, A, B\}$.

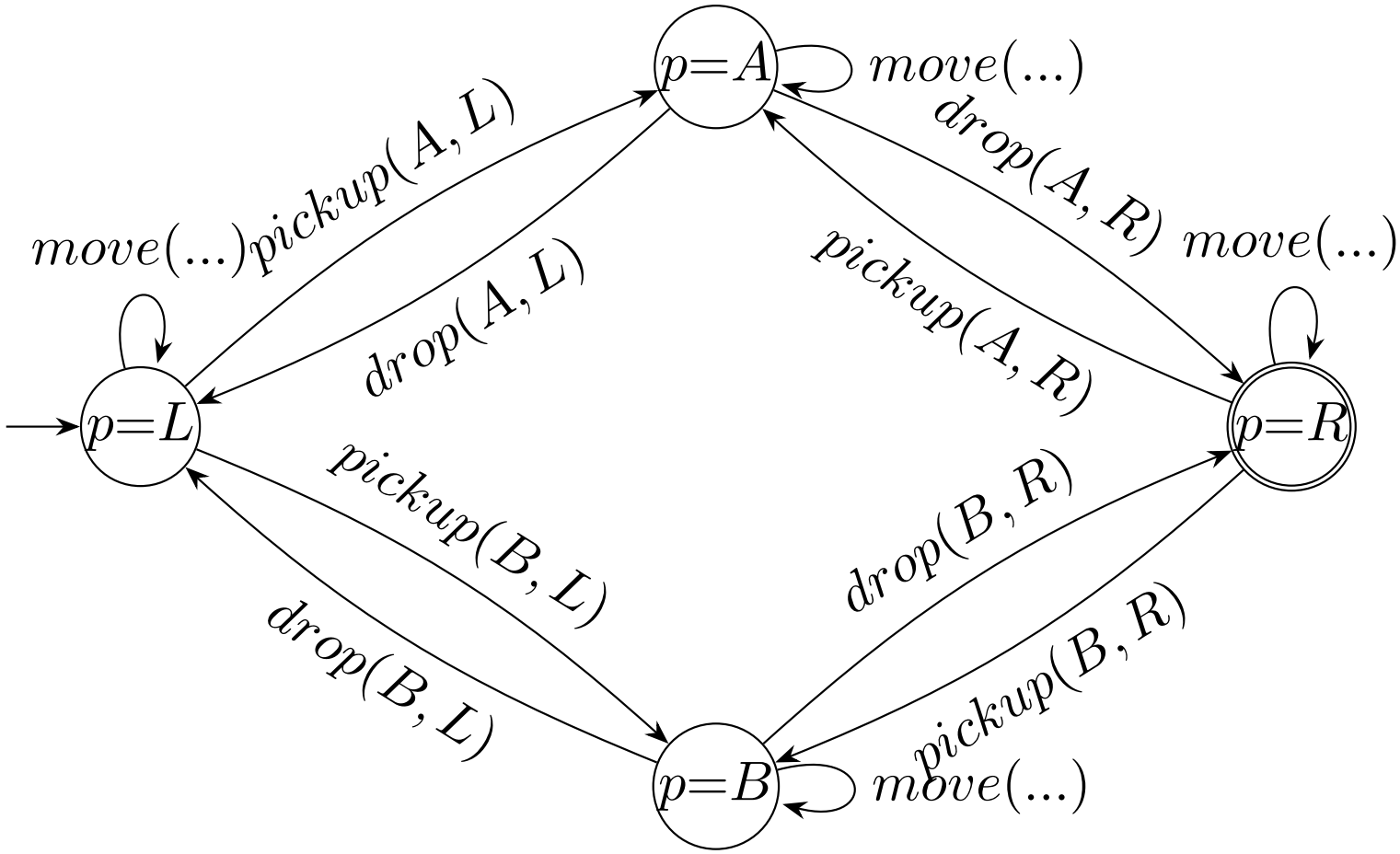
$$\begin{aligned}\mathcal{A} = &\{pickup(x, y) \mid x \in \{A, B\}, y \in \{L, R\}\}\\ &\cup \{drop(x, y) \mid x \in \{A, B\}, y \in \{L, R\}\}\\ &\cup \{move(x, y, y') \mid x \in \{A, B\}, y, y' \in \{L, R\}, y \neq y'\}, \textit{with}\end{aligned}$$

- $\text{pre}_{pickup(x,y)}$: $p = y$; $\text{eff}_{pickup(x,y)}$: $p = x$;
- $\text{pre}_{drop(x,y)}$: $p = x$; $\text{eff}_{drop(x,y)}$: $p = y$;
- $\text{pre}_{move(x,y,y')}$: $\emptyset$; $\text{eff}_{move(x,y,y')}$: $\emptyset$.

$I = \{p = L\}$. $G = \{p = R\}$.

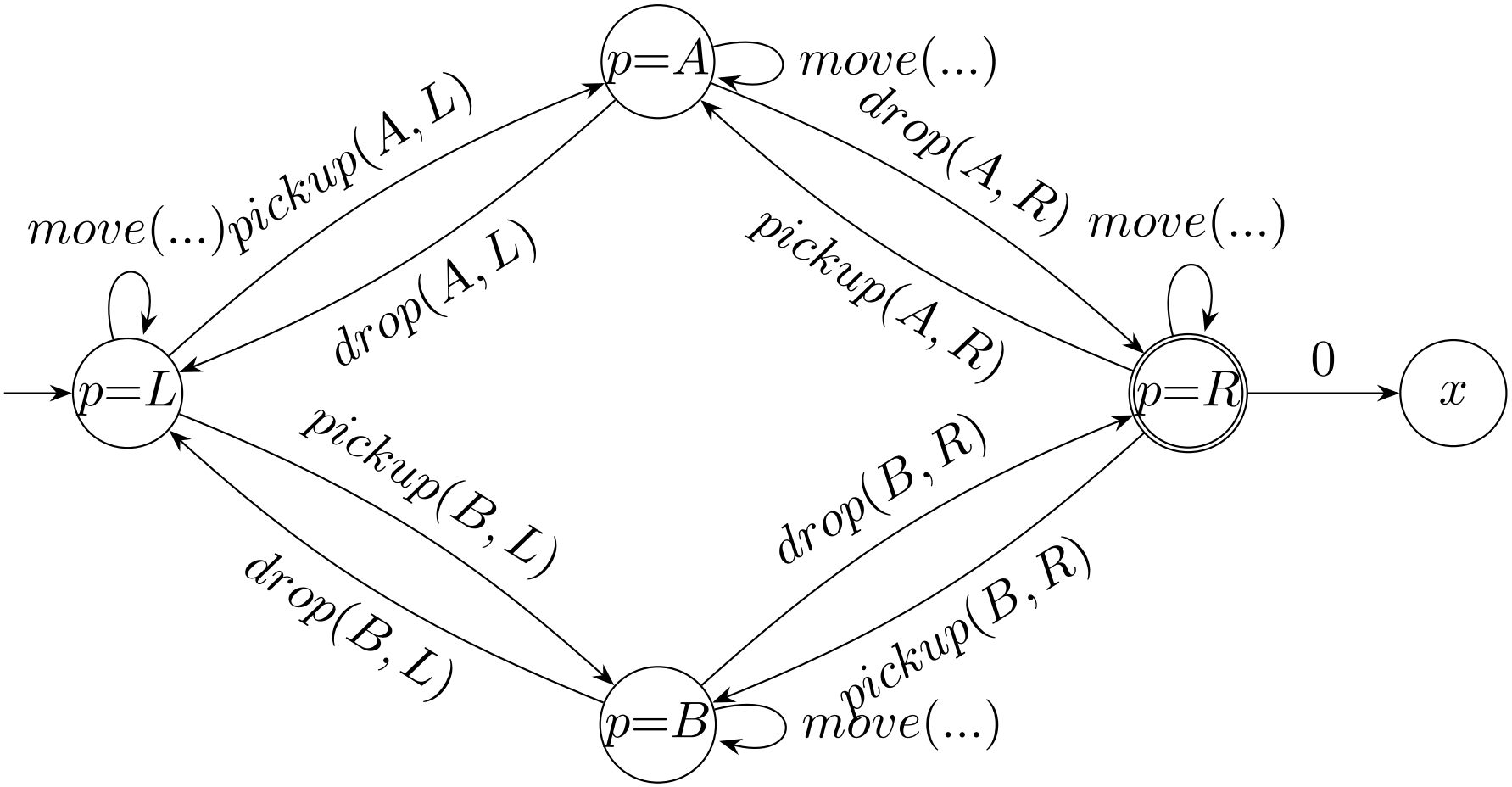
(I) Example

$\Theta_{\Pi|_P}$



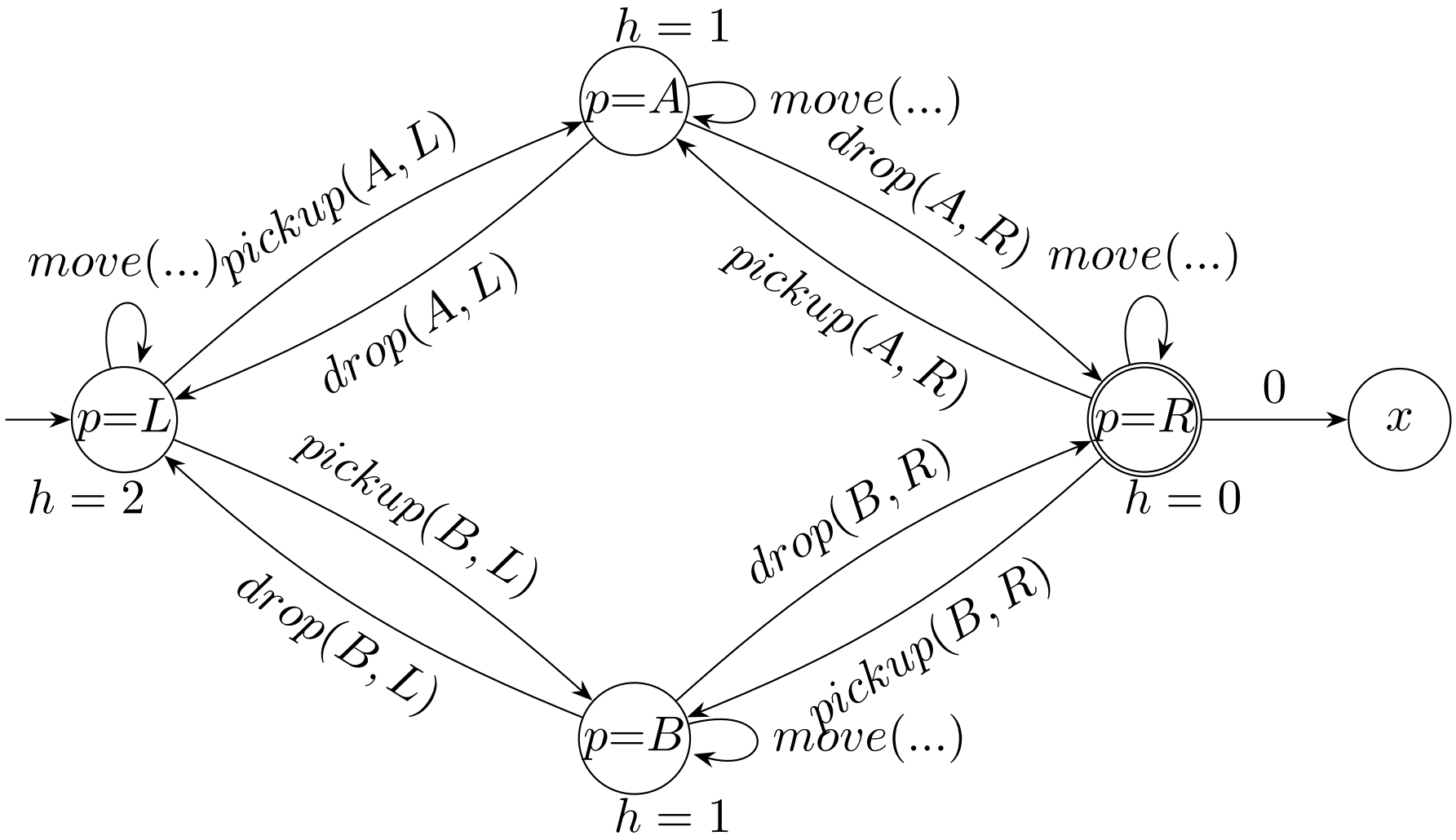
(I) Example

$\Theta_{\Pi|_P}$ with added x



(I) Example

$\Theta_{\Pi|_P}$ with added x and computed $h^*_{\Theta_{\Pi|_P}}$



(II) Lookup Step: Overview

Basic observations and method:

- During search, we do not need the actual abstract state space (transitions etc): The PDB is the only piece of information necessary to represent h^P.

  $\rightsquigarrow$ We can throw away the abstract state space Θ' once the PDB is computed.

  $\rightsquigarrow$ Space requirement for the PDB heuristic during search is linear in number of abstract states S': *PDB* has one table entry for each abstract state.

- Design a perfect hash function mapping projected states $s|_P$ to numbers in the range $\{0, \dots, |S'| - 1\}$.

  $\rightsquigarrow$ Index a table *PDB* by these hash values. Given a state s during search, to compute $h^P(s)$, map $\pi_P(s) = s|_P$ to its hash value and lookup the table entry of *PDB*.

(II) Lookup Step: Here's How To [For Reference Only]

Perfect hash function ≈ numeral system over variable domains:

- Let $P = \{v_1, \ldots, v_k\}$ be the pattern.
- Assume wlog that all variable domains are natural numbers counted from 0, i.e., $D_v = \{0, 1, \ldots, |D_v| - 1\}$.
- For all $i \in \{1, \ldots, k\}$, we precompute $N_i := \prod_{j=1}^{i-1} |D_{v_j}|$.

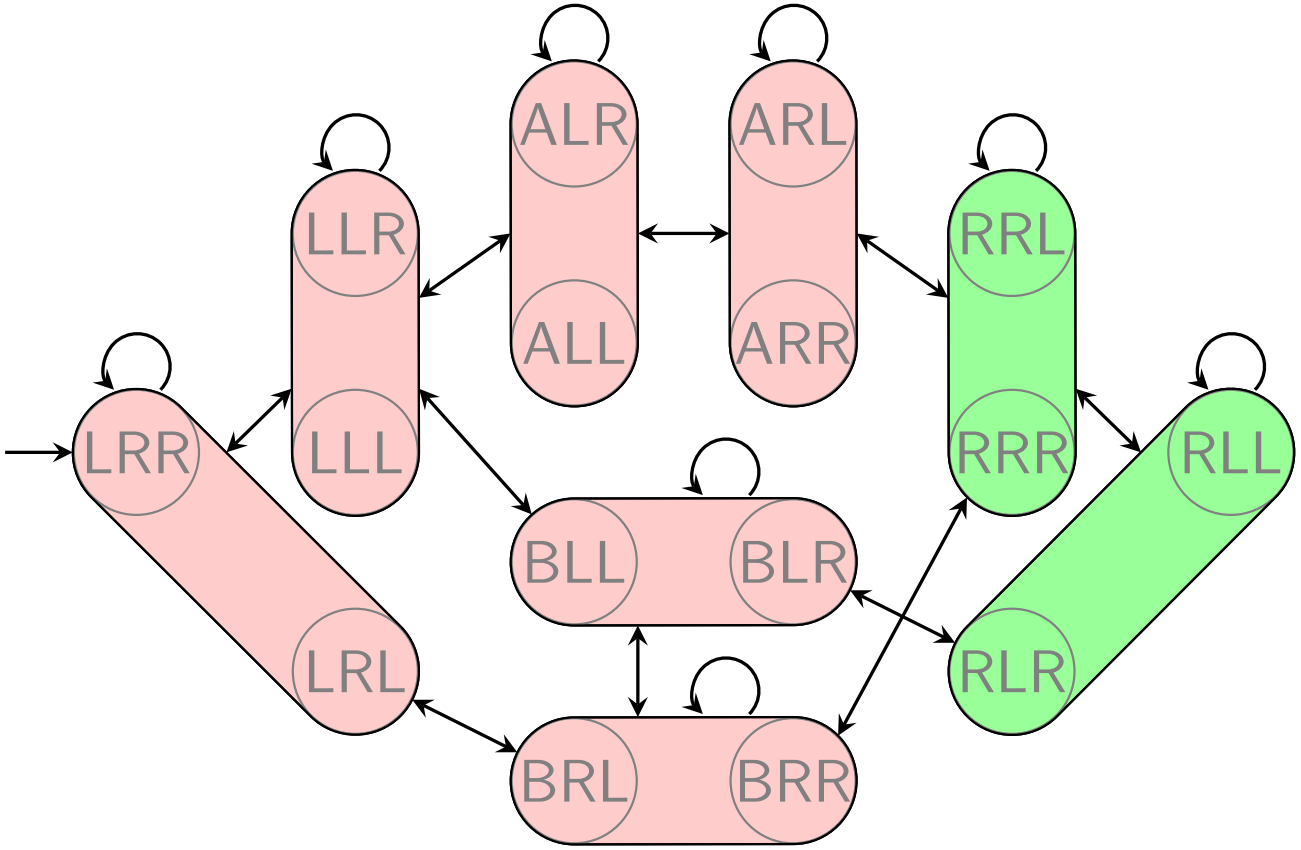
Looking Up a Pattern Database Heuristic Value

function $PDBheuristic(s)$:
  index $:= \sum_{i=1}^{k} N_i * s(v_i)$
  return *PDB*[*index*]

Note: This lookup runs in time and space $O(k)$. This is *very* fast. For comparison, delete-relaxation heuristics need time $O(\|\Pi\|)$ per state.

(II) Example: "Logistics mal anders" [For Reference Only]

Abstraction induced by $\pi_{\{\text{package},\text{truck A}\}}$:



(II) Example [For Reference Only]

Pattern variables and domains:

- $P = \{v_1, v_2\}$ with $v_1 =$ package, $v_2 =$ truck A.
- $D_{v_1} = \{L, R, A, B\} \approx \{0, 1, 2, 3\}$
- $D_{v_2} = \{L, R\} \approx \{0, 1\}$

$\rightsquigarrow N_1 = \prod_{j=1}^{0} |D_{v_j}| = 1.$

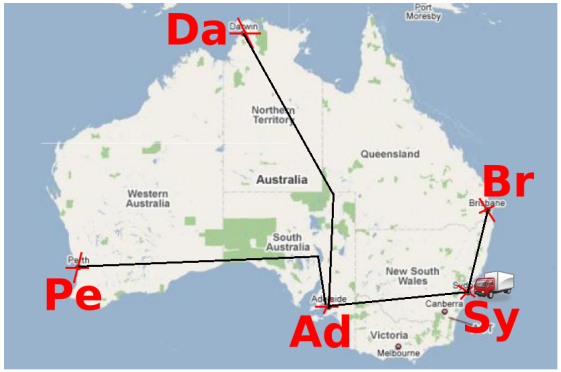
$\rightsquigarrow N_2 = \prod_{j=1}^{1} |D_{v_j}| = 4.$

$\rightsquigarrow$ *index*$(s) = 1 * s($package$) + 4 * s($truck A$)$.

$\rightsquigarrow$ **Pattern database:**

abstract state	LL	RL	AL	BL	LR	RR	AR	BR
index	0	1	2	3	4	5	6	7
value	2	0	2	1	2	0	1	1

(II) Another Example [For Reference Only]



- Variables: $at : \{Sy, Ad, Br, Pe, Ad\}$; $v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Ad\}$.
- Actions: $drive(x, y)$ where x, y have a road.
- Costs: $Sy \leftrightarrow Br : 1$, $Sy \leftrightarrow Ad : 1.5$, $Ad \leftrightarrow Pe : 3.5$, $Ad \leftrightarrow Da : 4$.
- Initial state: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
- Goal: $at = Sy, v(x) = T$ for all x.

$\rightsquigarrow$ Say our pattern P is $\{v_1 = v(Br), v_2 = v(Pe), v_3 = v(Da)\}$. What is the PDB?

$\rightsquigarrow$ $D_{v(Br)} = \{F, T\} \approx \{0, 1\}$, $N_1 = 1$; $D_{v(Pe)} = \{F, T\} \approx \{0, 1\}$, $N_2 = 2$; $D_{v(Da)} = \{F, T\} \approx \{0, 1\}$, $N_3 = 4$.

abstract state	FFF	TFF	FTF	TTF	FFT	TFT	FTT	TTT
index	0	1	2	3	4	5	6	7
value	8.5	7.5	5	4	4.5	3.5	1	0

How to *Admissibly* Combine Multiple Abstractions?

Maximizing over several abstractions:

- Each abstraction mapping gives rise to an admissible heuristic.
- By computing the maximum of several admissible heuristics, we obtain another admissible heuristic which dominates these.
- Thus, we can always compute several abstractions and maximize over the individual abstract goal distances.
- Yawn.

Better idea: Summing over several abstractions!

- In some cases, the abstraction heuristics are additive sum and still remain admissible.
- Summation often leads to much higher estimates than maximization, so it is important to understand when abstractions are additive.

Additive Abstractions: Example 15-Puzzle

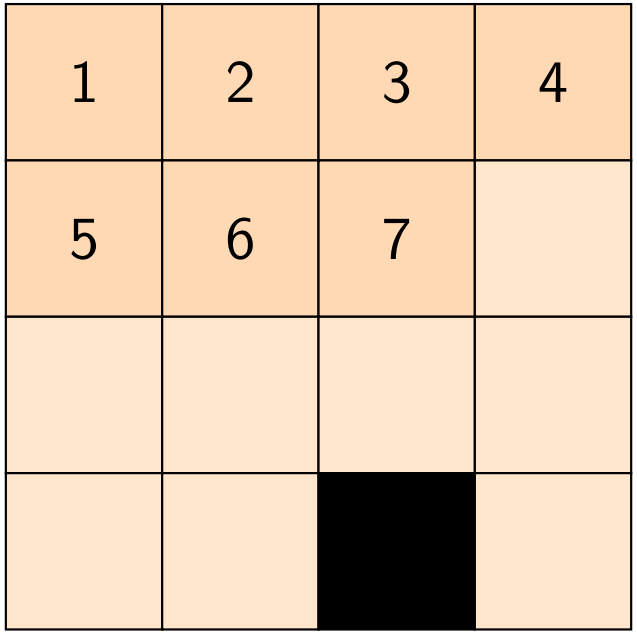
	2		6
5	7		
3	4	1	

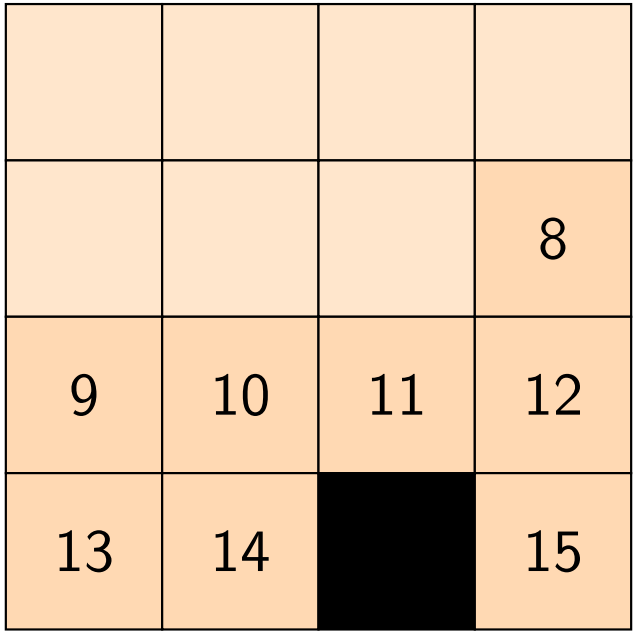
9		12	
		14	13
			11
15	10	8	

- 1st abstraction: Ignore location of $8, \ldots, 15$
- 2nd abstraction: Ignore location of $1, \ldots, 7$

$\rightsquigarrow$ Is the sum of the abstraction heuristics admissible?

Additive Abstractions: Example 15-Puzzle





- 1st abstraction: Ignore location of $8, \ldots, 15$
- 2nd abstraction: Ignore location of $1, \ldots, 7$

$\rightsquigarrow$ Is the sum of the abstraction heuristics admissible? No

Additive Abstractions: Example 15-Puzzle

1	2	3	4
5	6	7	

			8
9	10	11	12
13	14		15

- 1st abstraction: Ignore location of $8, \ldots, 15$ and blank.
- 2nd abstraction: Ignore location of $1, \ldots, 7$ and blank.

$\rightsquigarrow$ Is the sum of the abstraction heuristics admissible?

Additive Abstractions: Example 15-Puzzle

1	2	3	4
5	6	7	

			8
9	10	11	12
13	14		15

- 1st abstraction: Ignore location of $8, \ldots, 15$ and blank.
- 2nd abstraction: Ignore location of $1, \ldots, 7$ and blank.

$\rightsquigarrow$ Is the sum of the abstraction heuristics admissible? Yes

Orthogonal Abstractions

Terminology: If $s = t$ in (s, a, t), then the transition is called a self-loop.

Definition (Actions Affecting an Abstraction)

Let α be an abstraction of Θ, and let a be one of the actions in Θ. We say that a affects α if Θ^α has at least one non-self-loop transition induced by a, i.e., if there exists a transition $(\alpha(s), a, \alpha(t))$ with $\alpha(s) \neq \alpha(t)$.

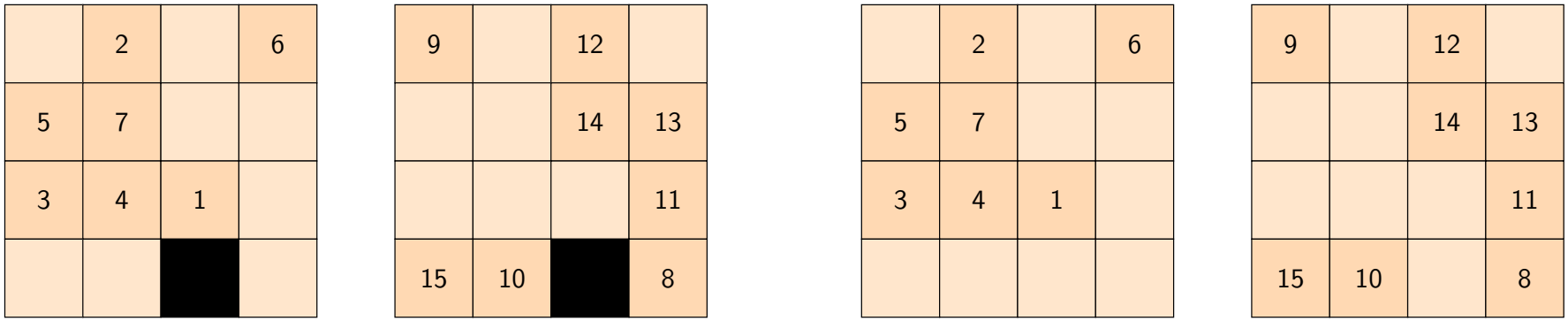
$\rightsquigarrow$ Here is a simple sufficient criterion for additivity:

Definition (Orthogonal Abstractions)

Let α_1 and α_2 be abstractions of Θ. We say that α_1 and α_2 are orthogonal if no action of Θ affects both α_1 and α_2.

Orthogonal Abstractions: Example 15-Puzze

Reminder: An action affects α if it induces a non-self loop transition in Θ^α. We say that α_1 and α_2 are orthogonal if no action of Θ affects both α_1 and α_2.



$\rightsquigarrow$ Are the left-hand side abstraction mappings α_{left} and α_{right} orthogonal? No. E.g., consider the action that moves the blank to the right here, mapping the current state s to state t. This transition is not a self-loop in either of the two abstractions: $\alpha_{\text{left}}(s) \neq \alpha_{\text{left}}(t)$ and $\alpha_{\text{right}}(s) \neq \alpha_{\text{right}}(t)$.

$\rightsquigarrow$ Are the right-hand side abstraction mappings α_{left} and α_{right} orthogonal? Yes. Say a is any action that affects α_{left}. Then a moves a tile t_i for $i \in \{1, \ldots, 7\}$. Neither that t_i nor the blank are accounted for in α_{right} so a induces only self-loops there. Same vice versa.

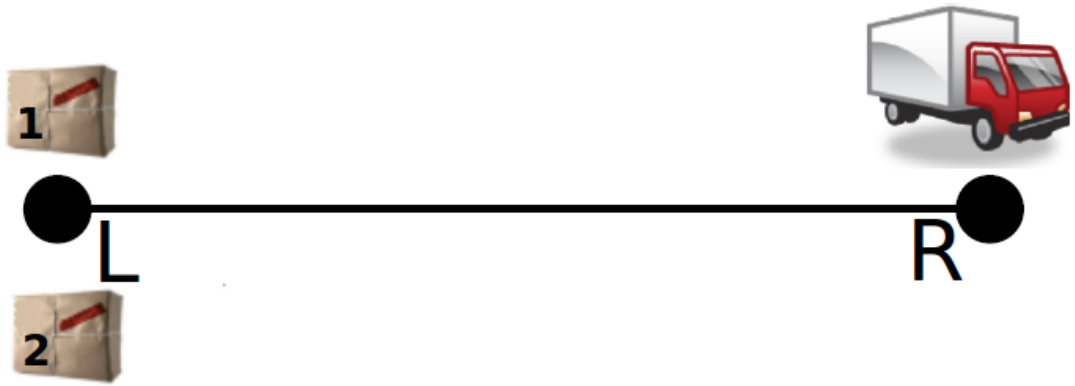
Orthogonality and Additivity

Theorem (Orthogonal Abstractions are Additive)

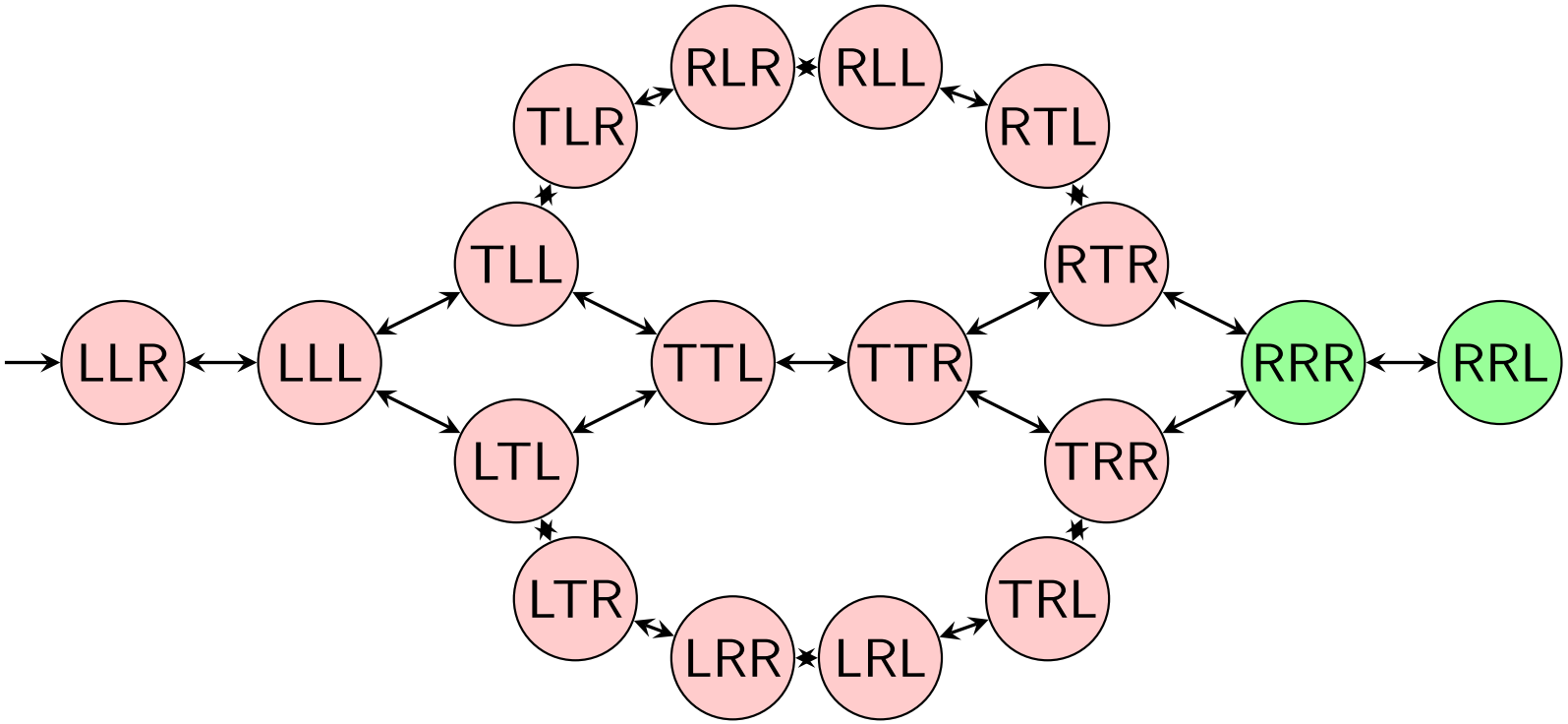
Let $\alpha_1, \dots, \alpha_n$ be pairwise orthogonal abstractions for the same state space Θ. Then $\sum_{i=1}^{n} h^{\alpha_i}$ is consistent, and thus also admissible and safe.

$\rightsquigarrow$ Intuition for admissibility: "Self-loops don't count." Every transition in an optimal solution path affects at most one of the abstractions, and thus is counted in at most one of the abstraction heuristics.

To illustrate the proof idea, we use yet another variant of "Logistics":

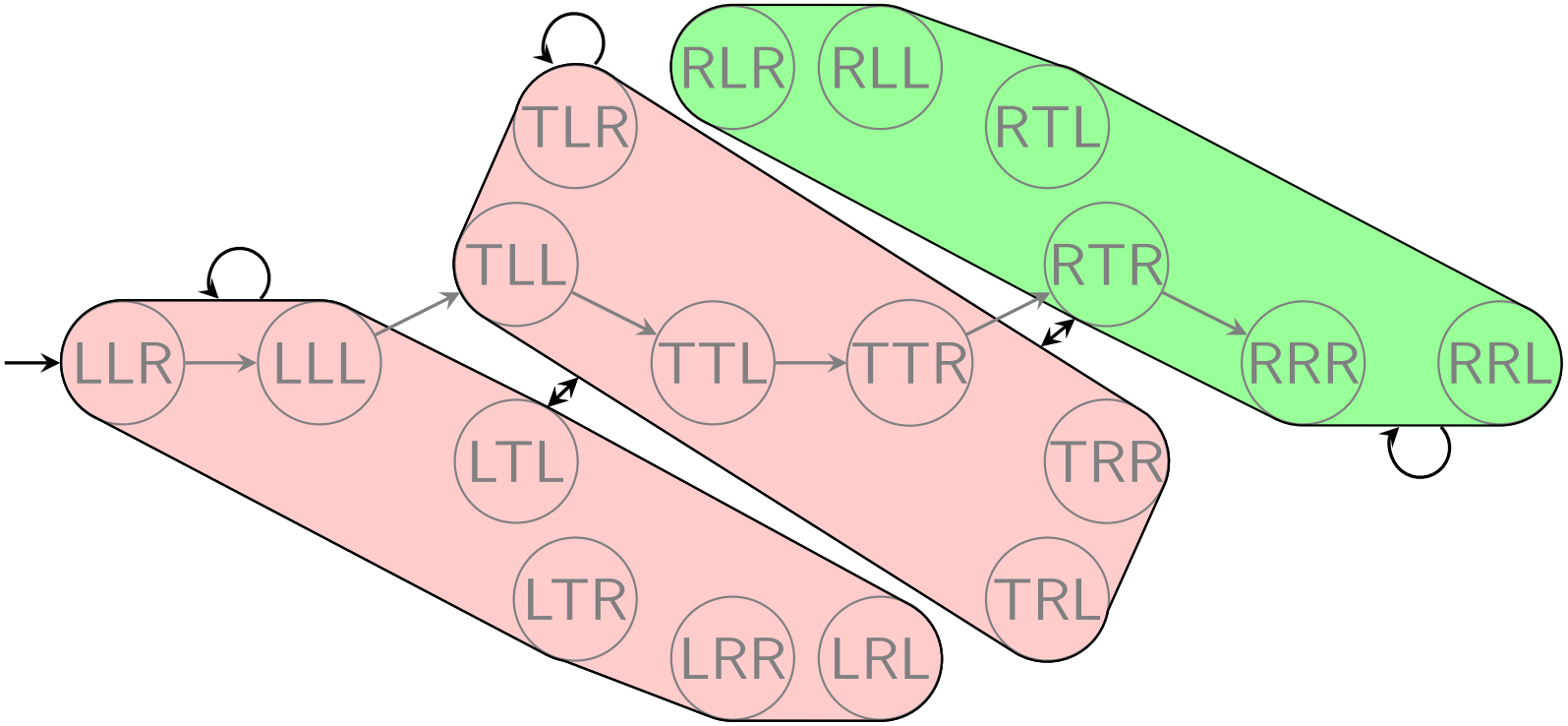


Orthogonality and Additivity: Illustration



State space Θ. State variables: package 1, package 2, truck.

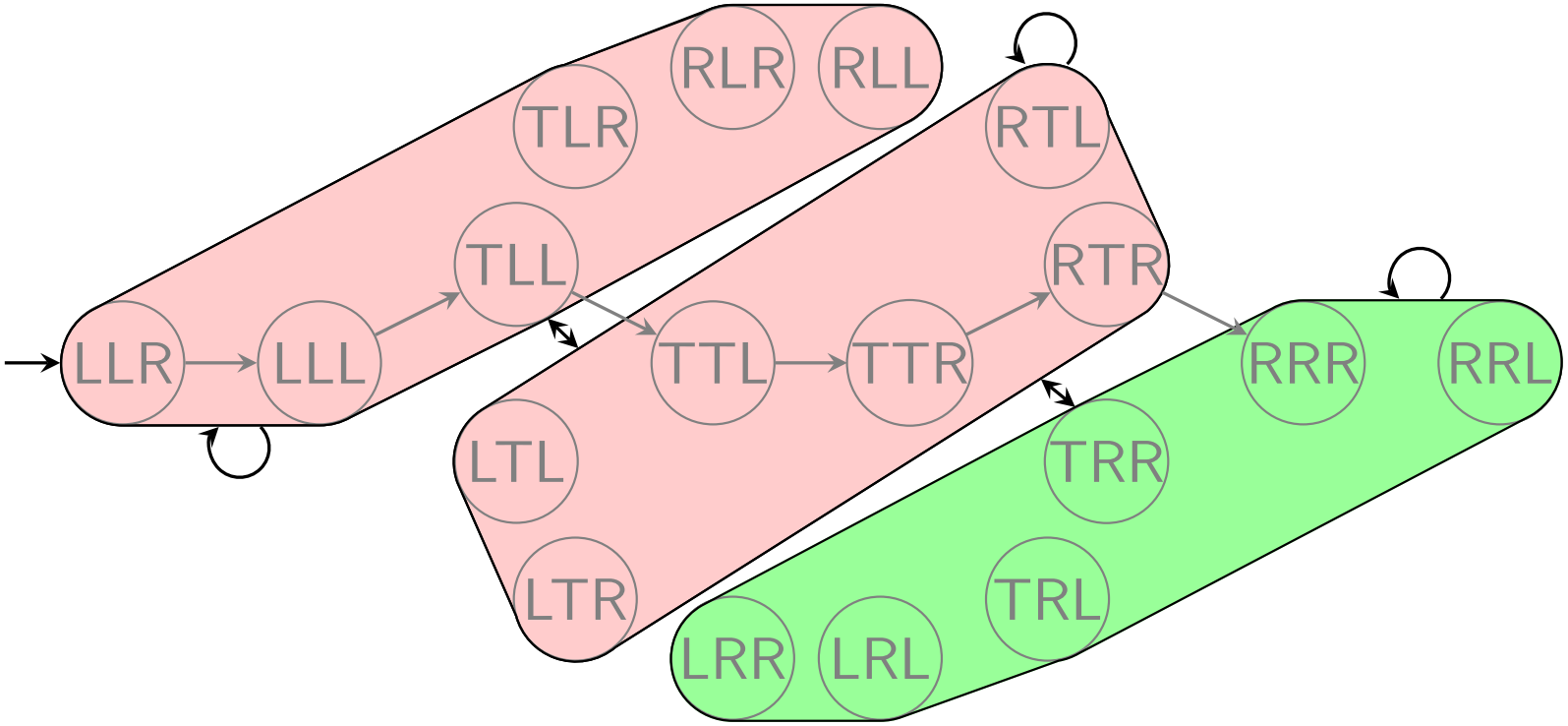
Orthogonality and Additivity: Illustration



Abstraction α_1.

Mapping: Only consider position of package 1.

Orthogonality and Additivity: Illustration



Abstraction α_2. (orthogonal to α_1)
Mapping: Only consider position of package 2.

Orthogonality and Additivity: Proof

Proof. Let $\Theta = (S, A, c, T, I, S^G)$.

$\sum_{i=1}^{n} h^{\alpha_i}(s) = 0$ for all $s \in \mathcal{S}_G$, because $h^{\alpha_i}(s) = 0$ for all $s \in \mathcal{S}_G$. So, $\sum_{i=1}^{n} h^{\alpha_i}(s)$ is a heuristic as per our definition.

For consistency, consider any state transition $(s, a, t) \in T$ in the concrete state space. We need to show that $\sum_{i=1}^{n} h^{\alpha_i}(s) \leq \sum_{i=1}^{n} h^{\alpha_i}(t) + c(a)$.

Because the abstraction mappings are orthogonal, $\alpha_i(s) \neq \alpha_i(t)$ for at most one $i \in \{1, \ldots, n\}$. (Assume the opposite were true, and there were $i \neq j \in \{1, \ldots, n\}$ s.t. $\alpha_i(s) \neq \alpha_i(t)$ and $\alpha_j(s) \neq \alpha_j(t)$. Then a induces a non-self-loop transition in both Θ^{α_i} and Θ^{α_j}, and thus α_i and α_j are not orthogonal, in contradiction.)

Orthogonality and Additivity: Proof, ctd.

Situation: Consider a concrete state transition $(s, a, t) \in T$. We need to show that $\sum_{i=1}^{n} h^{\alpha_i}(s) \leq \sum_{i=1}^{n} h^{\alpha_i}(t) + c(a)$.
We know that $\alpha_i(s) \neq \alpha_i(t)$ for at most one $i \in \{1, \ldots, n\}$.

Case 1: $\alpha_i(s) = \alpha_i(t)$ for all i. **Then:**

$$\begin{aligned}
\sum_{i=1}^{n} h^{\alpha_i}(s) &= \sum_{i=1}^{n} h^*_{\Theta^{\alpha_i}}(\alpha_i(s)) \\
&= \sum_{i=1}^{n} h^*_{\Theta^{\alpha_i}}(\alpha_i(t)) \text{ [because } \alpha_i(s) = \alpha_i(t)\text{]} \\
&= \sum_{i=1}^{n} h^{\alpha_i}(t) \\
&\leq \sum_{i=1}^{n} h^{\alpha_i}(t) + c(a).
\end{aligned}$$

Case 2: $\alpha_k(s) \neq \alpha_k(t)$, and $\alpha_i(s) = \alpha_i(t)$ for $i \neq k$. **Then:**

$$\begin{aligned}
\sum_{i=1}^{n} h^{\alpha_i}(s) &= \sum_{i \neq k} h^*_{\Theta^{\alpha_i}}(\alpha_i(s)) + h^{\alpha_k}(s) \\
&= \sum_{i \neq k} h^*_{\Theta^{\alpha_i}}(\alpha_i(t)) + h^{\alpha_k}(s) \text{ [} \alpha_i(s) = \alpha_i(t) \text{ for } i \neq k\text{]} \\
&\leq \sum_{i \neq k} h^*_{\Theta^{\alpha_i}}(\alpha_i(t)) + h^{\alpha_k}(t) + c(a) \text{ [} h^{\alpha_k} \text{ is consistent]} \\
&= \sum_{i=1}^{n} h^{\alpha_i}(t) + c(a).
\end{aligned}$$

Pattern Collections

Pattern Collections? Why and How?

- The space requirements for a pattern database grow exponentially with the number of state variables in the pattern.

  $\rightsquigarrow$ Hence P must be small, severely limiting the usefulness of single-PDB heuristics h^P for large planning tasks.

- To overcome this limitation, planners using pattern databases can work with collections of multiple patterns.

- Given heuristics h^{P_1} and h^{P_2}, we can always get an admissible heuristic dominating each of h^{P_1} and h^{P_2} by $\max\{h^{P_1}, h^{P_2}\}$.

- Combination of h^{P_1} and h^{P_2} that would be much preferable: $h^{P_1} + h^{P_2}$.

  $\rightsquigarrow$ But, **for this to be admissible, h^{P_1} and h^{P_2} must be additive.**

Orthogonal Patterns

Lemma (Actions Affecting PDBs)

Let P be a pattern for an FDR planning task Π, and let a be an action in Π. Then a affects π_P if and only if there exists a variable $v \in P$ on which eff_a is defined.

Orthogonal Patterns

Lemma (Actions Affecting PDBs)

Let P be a pattern for an FDR planning task Π, and let a be an action in Π. Then a affects π_P if and only if there exists a variable $v \in P$ on which eff_a is defined.

Proof. Consider the syntactic projection $\Pi|_P$.
Only if: If a has no effect on P then its effect in $\Pi|_P$ is empty so it can induce only self-loops.
If: Design a state s in $\Pi|_P$ by projecting pre_a onto P and filling up the remaining P-values to be arbitrary but different from eff_a. Then a induces the non-self-loop transition $(s, a, s[\![a]\!])$ in $\Pi|_P$ (as $\mathrm{pre}_a(v) \neq \mathrm{eff}_a(v)$ whenever both are defined).

Orthogonal Patterns

Lemma (Actions Affecting PDBs)

Let P be a pattern for an FDR planning task Π, and let a be an action in Π. Then a affects π_P if and only if there exists a variable $v \in P$ on which eff_a is defined.

Theorem (Orthogonal Patterns)

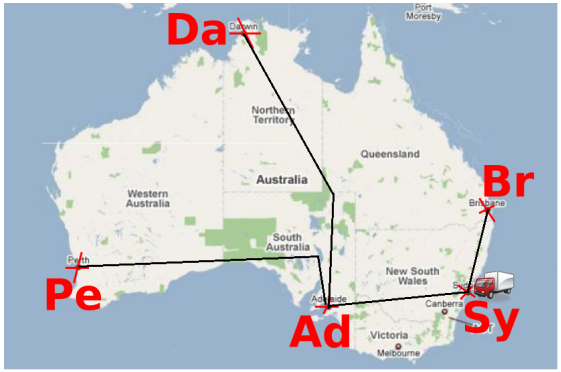
Let P_1 and P_2 be patterns for an FDR planning task Π. Then π_{P_1} and π_{P_2} are orthogonal if and only if there exists no action a in Π such that eff_a is defined for variables v, v' where $v \in P_1$ and $v' \in P_2$, i.e., $\mathcal{V}(\mathrm{eff}_a) \cap P_1 \cap P_2 \neq \emptyset$.

Terminology: In this situation, we also call the patterns P_1 and P_2 themselves (as opposed to π_{P_1} and π_{P_2}) orthogonal.

On orthogonality and (non)-intersecting patterns:

- If $P_1 \cap P_2 \neq \emptyset$, can P_1 and P_2 be orthogonal? No, except if all $v \in P_1 \cap P_2$ are not affected by any action, which is pathological.
- If $P_1 \cap P_2 = \emptyset$, are P_1 and P_2 necessarily orthogonal? No, there may be an action with a "cross-effect", on one variable from each.

Orthogonality in the Australia Example



- Variables: $at : \{Sy, Ad, Br, Pe, Ad\}$; $v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Ad\}$.
- Actions: $drive(x, y)$ where x, y have a road.
- Costs: $Sy \leftrightarrow Br : 1$, $Sy \leftrightarrow Ad : 2$, $Ad \leftrightarrow Pe : 3$, $Ad \leftrightarrow Da : 4$.
- Initial state: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
- Goal: $at = Sy, v(x) = T$ for all x.

Observe: Are the patterns $P_1 = \{v(Br)\}$ and $P_2 = \{v(Pe), v(Da)\}$ orthogonal?

$\rightsquigarrow$ Yes: Every action affects at most one of them.

Observe: Are the patterns $P_1' = \{v(Br), at\}$, $P_2 = \{v(Pe), v(Da)\}$ orthogonal?

$\rightsquigarrow$ No: E.g., driving to Perth affects both of them.

Orthogonal Patterns and Additivity

Terminology: Instead of "collection of pairwise orthogonal patterns", we also say orthogonal pattern collection or orthogonal collection.

Theorem (Orthogonal Patterns are Additive)

Let $\{P_1, \dots, P_k\}$ be an orthogonal pattern collection for an FDR planning task. Then $\sum_{i=1}^{k} h^{P_i}$ is consistent and therefore also admissible and safe.

Proof. The abstractions $\pi_{P_1}, \dots, \pi_{P_k}$ are pairwise orthogonal. The rest follows from Theorem "Orthogonal Abstractions are Additive"

Theorem

Let $\{P_1, \dots, P_k\}$ be an orthogonal pattern collection for an FDR planning task, and let P be a pattern such that $P_i \subseteq P$ for all $i \in \{1, \dots, k\}$.
Then $\sum_{i=1}^{k} h^{P_i} \leq h^P$.

Proof. Each π_{P_i} is a projection of $\Pi|_P$. Clearly, $\{P_1, \dots, P_k\}$ is an orthogonal pattern collection in $\Pi|_P$ as well. The rest follows from admissibility of $\sum_{i=1}^{k} h^{P_i}$ in $\Pi|_P$.

How to Exploit Orthogonal Patterns?

Given pattern collection $\mathcal{C}$: How to get the best possible lower bound?

1. Build the compatibility graph for $\mathcal{C}$.
   - The vertices are the patterns $P \in \mathcal{C}$.
   - Arcs between pairs of orthogonal patterns.
2. Compute all maximal cliques (maximal orthogonal sub-collections). (Computing maximal cliques is NP-hard, but this graph will typically be small.)

Abstract Example

FDR task with $V = \{v_1, v_2, v_3\}$, pattern collection $\mathcal{C} = \{P_1, \dots, P_4\}$ with $P_1 = \{v_1, v_2\}$, $P_2 = \{v_1\}$, $P_3 = \{v_2\}$ and $P_4 = \{v_3\}$.
There are actions affecting each individual variable, and the only actions affecting several variables affect v_1 and v_3.
$\rightsquigarrow$ Maximal cliques in the compatibility graph for $\mathcal{C}$? $\{P_1\}$, $\{P_2, P_3\}$, $\{P_3, P_4\}$.

The Cliques Heuristic

Definition (Cliques Heuristic)

Let Π be an FDR planning task, let $\mathcal{C}$ be a pattern collection for Π, and let $\mathrm{cliques}(\mathcal{C})$ be the set of all maximal cliques in the compatibility graph for $\mathcal{C}$. Then the cliques heuristic $h^{\mathcal{C}}$ for $\mathcal{C}$ is defined as

$$h^{\mathcal{C}}(s) = \max_{\mathcal{D} \in \mathrm{cliques}(\mathcal{C})} \sum_{P \in \mathcal{D}} h^P(s)$$

$\rightsquigarrow$ The cliques heuristic maximizes over all largest orthogonal subsets of our pattern collection. It is consistent and goal-aware.

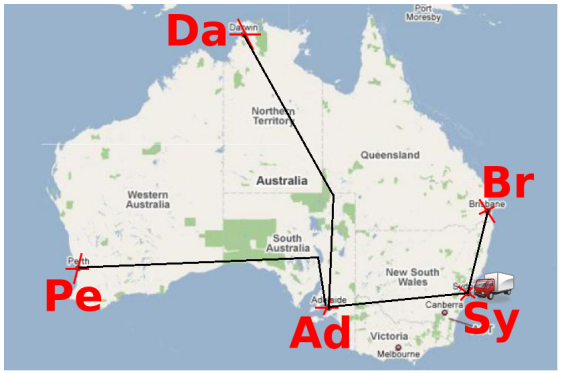
Same Abstract Example

Maximal cliques in the compatibility graph: $\{P_1\}$, $\{P_2, P_3\}$, $\{P_3, P_4\}$.

$\rightsquigarrow$ Cliques heuristic $h^{\mathcal{C}}$:

$$\begin{aligned} h^{\mathcal{C}} &= \max\{h^{P_1}, h^{P_2} + h^{P_3}, h^{P_3} + h^{P_4}\} \\ &= \max\{h^{\{v_1,v_2\}}, h^{\{v_1\}} + h^{\{v_2\}}, h^{\{v_2\}} + h^{\{v_3\}}\} \end{aligned}$$

The Australia Example Again



- Variables: $at : \{Sy, Ad, Br, Pe, Ad\}$;
  $v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Ad\}$.
- Actions: $drive(x, y)$ where x, y have a road.
- Costs: $Sy \leftrightarrow Br : 1$, $Sy \leftrightarrow Ad : 2$, $Ad \leftrightarrow Pe : 3$, $Ad \leftrightarrow Da : 4$.
- Initial state: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
- Goal: $at = Sy, v(x) = T$ for all x.

Observations:

- Say $P_1 = \{v(Br)\}$ and $P_2 = \{v(Pe), v(Da)\}$. What are the values of $h^{P_1}(I)$ and $h^{P_2}(I)$? $h^{P_1}(I) = 1$, $h^{P_2}(I) = 4 + 3 = 7$ (cf. slide 40).
- Say $\mathcal{C} = \{P_1, P_2\}$. What is the value of $h^{\mathcal{C}}(I)$? P_1 and P_2 are orthogonal (slide 52), so $h^{\mathcal{C}} = h^{P_1}(I) + h^{P_2}(I) = 1 + 7 = 8$.
- Say $P_1' = \{v(Br), at\}$. What is the value of $h^{P_1'}(I)$? $h^{P_1'}(I) = 2$ because now we have to drive back from Br to satisfy the truck goal.
- Say $\mathcal{C} = \{P_1', P_2\}$. What is the value of $h^{\mathcal{C}}(I)$? P_1' and P_2 are *not* orthogonal (slide 52), so $h^{\mathcal{C}} = \max\{h^{P_1'}(I), h^{P_2}(I)\} = 7$.

Note: Even though P_1' itself yields a better lower bound than P_1, replacing P_1 with P_1' is detrimental for the overall lower bound $h^{\mathcal{C}}(I)$.

Quiz: The Attack of the Zombie Tomatoes

- Goal: A and B both true.
- Initial state: A and B both false.
- Actions: $carA$ effect A cost 2; $carB$ effect B cost 2; $fancyCar$ effect A and B cost 3.
- Patterns $P_1 = \{A\}$ and $P_2 = \{B\}$.

Quiz

Are P_1 and P_2 orthogonal? What is the value of $h^{P_1}(I)$ and $h^{P_2}(I)$?

$\rightsquigarrow$ They are not orthogonal as $fancyCar$ affects both patterns. $h^{P_1}(I) = h^{P_2}(I) = 2$ due to $carA$ respectively $carB$.

Quiz

So $h^{\mathcal{C}}(I) =$

$\rightsquigarrow$ As P_1 and P_2 not orthogonal, $h^{\mathcal{C}}(I) = \max(h^{P_1}(I), h^{P_2}(I)) = 2$.

On the Power of the Cliques Heuristic

$\rightsquigarrow$ The cliques heuristic $h^{\mathcal{C}}$ is the best possible admissible heuristic we can derive from $\mathcal{C}$ based on exploiting only orthogonality.

In The Attack of the Zombie Tomatoes, that is not enough:

- Goal: A and B both true.
- Initial state: A and B both false.
- Actions: $carA$ effect A cost 2; $carB$ effect B cost 2; $fancyCar$ effect A and B cost 3.
- Patterns $P_1 = \{A\}$ and $P_2 = \{B\}$.

$\rightsquigarrow h^{\mathcal{C}}(I) = 2 < 3 = h^*(I)$.

$\rightsquigarrow$ Cost partitioning is a more general technique for combining heuristics, which always dominates $h^{\mathcal{C}}$, and which yields the perfect lower bound 3 in this particular example.

Summary

- An abstraction α is a surjective function on a state space Θ (e.g., of a planning task).
- h^* in Θ^α is the abstraction heuristic h^α, which is safe, admissible, and consistent.
- The heuristics of orthogonal abstractions are additive, i.e., their sum is admissible
- Pattern database (PDB) heuristics are abstraction heuristics based on projection to a subset of variables: the pattern. For FDR tasks, they can easily be implemented via syntactic projection on the task representation.
- Pattern databases are lookup tables that store heuristic values, indexed by perfect hash values for projected states.
- Two patterns are orthogonal if no action affects variables from both. The heuristics are then additive, i.e., their sum is admissible.
- Given a pattern collection, the cliques heuristic sums heuristic values from all orthogonal pattern subsets, and maximizes over these sums.

Reading

- *Planning with Pattern Databases* [Edelkamp (2001)].

  Available at:

  `http://www.tzi.de/~edelkamp/publications/drafts/patternPlan.pdf`

  Content: The first paper introducing pattern database heuristics to planning. Formulated in the STRIPS setting, where a PDB is defined as a subset of facts. Contains the corresponding version of orthogonality, and empirical results on the benchmarks at the time.

Reading, ctd.

- *Domain-Independent Construction of Pattern Database Heuristics for Cost-Optimal Planning* [Haslum *et al.* (2007)].

  Available at:

  `http://users.cecs.anu.edu.au/~patrik/publik/patterns3.pdf`

  Content: Method for automatically constructing and using collections of patterns, formulated in FDR (well, in STRIPS as well as its correspondence to "multi-valued state variables"). Main basis of this chapter. Introduces the cliques heuristic. Empirical results in 15-puzzle, Sokoban, and Logistics.

References I

Joseph C. Culberson and Jonathan Schaeffer. Pattern databases. *Computational Intelligence*, 14(3):318–334, 1998.

Klaus Dräger, Bernd Finkbeiner, and Andreas Podelski. Directed model checking with distance-preserving abstractions. In Antti Valmari, editor, *Proceedings of the 13th International SPIN Workshop (SPIN 2006)*, volume 3925 of *Lecture Notes in Computer Science*, pages 19–34. Springer-Verlag, 2006.

Stefan Edelkamp. Planning with pattern databases. In *Proceedings of the 6th European Conference on Planning (ECP'01)*, pages 13–24, 2001.

Patrik Haslum, Adi Botea, Malte Helmert, Blai Bonet, and Sven Koenig. Domain-independent construction of pattern database heuristics for cost-optimal planning. In *Proceedings of the 22nd National Conference of the American Association for Artificial Intelligence (AAAI'07)*, pages 1007–1012, 2007.

Malte Helmert, Patrik Haslum, and Jörg Hoffmann. Flexible abstraction heuristics for optimal sequential planning. In Mark Boddy, Maria Fox, and Sylvie Thiebaux, editors, *Proceedings of the 17th International Conference on Automated Planning and Scheduling (ICAPS'07)*, pages 176–183, Providence, Rhode Island, USA, 2007. Morgan Kaufmann.

References II

Malte Helmert, Patrik Haslum, Jörg Hoffmann, and Raz Nissim. Merge & shrink abstraction: A method for generating lower bounds in factored state spaces. *Journal of the Association for Computing Machinery*, 61(3):16.1–16.63, 2014.

István T. Hernádvölgyi and Robert C. Holte. Experiments with automatically created memory-based heuristics. In Berthe Y. Choueiry and Toby Walsh, editors, *Proceedings of the 4th International Symposium on Abstraction, Reformulation and Approximation (SARA 2000)*, volume 1864 of *Lecture Notes in Artificial Intelligence*, pages 281–290. Springer-Verlag, 2000.

Michael Katz, Jörg Hoffmann, and Malte Helmert. How to relax a bisimulation? In *Proceedings of the 22nd International Conference on Automated Planning and Scheduling (ICAPS'12)*, pages 101–109, 2012.

Jendrik Seipp and Malte Helmert. Counterexample-guided Cartesian abstraction refinement. In *Proceedings of the 23rd International Conference on Automated Planning and Scheduling (ICAPS'13)*, pages 347–351, 2013.